

Edge Computing Enabled Large-Scale Traffic Flow Prediction With GPT in Intelligent Autonomous Transport System for 6G Network

Yi Rong¹, Yingchi Mao¹, Huajun Cui, Xiaoming He, *Member, IEEE*, and Mingkai Chen², *Member, IEEE*

Abstract—The Intelligent Autonomous Transport System in 6G (6G-IATS) refers to the coordination of 6G, Artificial Intelligence (AI), and intelligent transportation systems, which is expected to revolutionize future intelligent transportation systems. In 6G-IATS, large-scale traffic flow prediction, affiliated with time series prediction, holds significant value for transportation planning and urban management. As an emerging AI method, Large Language Models (LLMs) have emerged prominently in time series forecasting. Unfortunately, it is challenging to achieve accurate and efficient large-scale traffic flow prediction by LLMs in 6G-IATS, due to the two issues: a) these LLMs fail to capture the spatio-temporal correlations in a large-scale road network, leading to limited prediction accuracy, and b) they process a substantial amount of training data on the central server, which imposes low training efficiency. Jointly considering the two concerns, this paper proposes a novel LLM and edge computing-based architecture for large-scale traffic flow prediction in 6G-IATS, called Spatio-Temporal Generative Large Language Model on Edge (STGLLM-E). In this architecture, we first decompose the entire large-scale road network into several subgraphs. To capture the spatio-temporal correlations, an LLM-based method named Spatio-Temporal Generative Large Language Model (STGLLM) including Spatio-Temporal Module (STM) and Generative Large Language Model (GLLM) is proposed. Secondly, to improve the training efficiency of the STGLLM-E, an edge training strategy based on edge servers is devised. Experiments are conducted on two real-world traffic flow datasets. The experimental results illustrate that the STGLLM-E is superior to the baselines in the prediction accuracy and the efficiency of training.

Index Terms—IATS, 6G, large-scale traffic flow prediction, LLMs, edge computing.

I. INTRODUCTION

WITH the emergence of mobile communication, we are ushering in an exceptional 6G era. 6G has the characteristic of low delay, high bandwidth, and extensive connectivity, giving rise to various application scenarios, such as intelligent transportation systems [1]. Moreover, thanks to the explosive evolution of 6G and Artificial Intelligence (AI), it is possible to make intelligent transportation systems more autonomous and efficient. Hence, the coordination of 6G, AI, and intelligent transportation systems namely Intelligent Autonomous Transport System in 6G (6G-IATS) has received widespread attention. 6G-IATS can utilize AI methods to effectively analyze large amounts of data collected from intelligent transportation system devices and then make rational decisions, intelligent traffic control, and adaptive resource allocation under the impetus of 6G. Especially in data analysis, AI methods have shown great potential.

In 6G-IATS applications, the internet of vehicles has become a hot topic in recent years [2], which enables vehicles to interact with edge servers to acquire valuable information. In this way, smart vehicles can quickly make reasonable decisions to enhance traffic efficiency based on the information. For this purpose, sensors have been deployed on the roads. The data from these sensors can be analyzed and mined to extract valuable information. One of the data analysis and mining tasks is large-scale traffic flow prediction, *i.e.*, forecasting future traffic flow in more than 1000 roads. Future road traffic flow Information can be rapidly transmitted to vehicles through 6G communication technologies for route planning and congestion alleviation [3]. Considering the powerful data analysis ability, leveraging AI methods is promising for large-scale traffic flow prediction in 6G-IATS.

In fact, the traffic flow has proven to be highly predictable [4]. Numerous efforts have focused on traffic flow prediction using AI methods [5], [6], [7], [8], [9]. Previous attempts explore the feasibility of classical statistical methods such as Autoregressive Integrated Moving Average (ARIMA) [5], and shallow machine learning methods like Support Vector Regression (SVR) [6]. Unfortunately, the modern transportation is sophisticated. Since a road network contains multiple interconnected roads, the traffic flow on each road is impacted

Manuscript received 15 May 2024; revised 10 August 2024; accepted 1 September 2024. Date of publication 17 September 2024; date of current version 21 October 2025. This work was supported in part by the Key Research and Development Program of China under Grant 2022YFC3005401; in part by the Key Research and Development Program of China, Yunnan Province, under Grant 202203AA080009; in part by the Technology Project of Huaneng Group under Grant HNKJ22-HF94 and Grant HNKJ20-H46; in part by the Key Research and Development Program of Jiangsu Province Key Project and Topics under Grant BE2023035; and in part by the National Natural Science Foundation for Young Scientists under Grant 62402246. The Associate Editor for this article was S. Mumtaz. (Corresponding authors: Mingkai Chen; Yingchi Mao.)

Yi Rong and Yingchi Mao are with the College of Computer Science and Software Engineering, Hohai University, Nanjing 210098, China (e-mail: rongyi1220@163.com; yingchimaomao@hhu.edu.cn).

Huajun Cui is with the Digital Intelligence Research Institute, PowerChina Beijing Engineering Corporation Ltd., Beijing 100013, China (e-mail: cuihuajun@bjy.powerchina.cn).

Xiaoming He is with the College of Internet of Things, Nanjing University of Posts and Telecommunications, Nanjing 210003, China (e-mail: hexiaoming@njupt.edu.cn).

Mingkai Chen is with the Key Laboratory of Broadband Wireless Communication and Sensor Network Technology, Nanjing University of Posts and Telecommunications, Nanjing 210003, China (e-mail: mkchen@njupt.edu.cn).

Digital Object Identifier 10.1109/TITS.2024.3456890

by its neighbors, regarding the spatial dependency. The future traffic flow in each road is substantially influenced by its past, which is denoted as the temporal dependency. These two types of dependencies are correlated. We think they are the spatio-temporal correlations. In addition, the temporal and spatial dependencies change all the time. These models cannot extract such spatio-temporal correlations in traffic flow data generated by a road network, leading to low prediction precision.

With the emergence of the deep learning technique, the various spatio-temporal models based on the Graph Convolutional Network (GCN) (*e.g.*, Diffusion Convolutional Recurrent Neural Network (DCRNN) [7], and Graph WaveNet (GWN) [8]) and the Transformer (*e.g.*, Graph Multi-Attention Network (GMAN) [9]) extract the spatio-temporal correlations. Although these GCN and Transformer models look promising, they do not perform well on a large-scale road network in 6G-IATS, *i.e.*, low prediction accuracy or low efficiency. Unfortunately, these GCN- and Transformer-based models only work on road networks with fewer than 350 roads. In detail, our objective is to implement large-scale traffic flow prediction, accurately and efficiently. Hence, the prediction accuracy and efficiency are evaluated on the road network with the same scale. As shown in Fig. 1 (a), we find that GCN-based models outperform Transformer-based models in prediction precision, but the prediction accuracy of the former remains unsatisfactory. The training efficiency of all models is generally low. The main reason for poor prediction accuracy is that the spatio-temporal correlations cannot be effectively extracted in a large-scale road network. There are two primary reasons for the low training efficiency, listed as: a) all models are trained on a center server. A large-scale road network brings exponentially increasing traffic flow data compared to road networks with fewer than 350 roads. Uploading these data to the center server may generate a huge computational load, and b) the existing GCN and Transformer models require traversing abundant neighbors of each road in the large-scale road network when capturing the spatial dependencies.

Recently, we have witnessed the advent of Large Language Models (LLMs), which have brought the leap to natural language processing [10]. As an emerging AI method, LLMs also have demonstrated outstanding performance in time series prediction [11], [12]. For instance, Zhou et al. designed a unified framework named Frozen Pretrained Transformer (FPT) that leverages pre-trained GPT-2 with frozen Multi-Head Self Attention (MHSA) layers and feedforward layers to perform time series prediction [11]. Chang et al. presented LLM4TS for seven widely multivariate time series prediction tasks through a two-phase fine-tuning strategy to LLMs [12]. The experimental results show that the LLM achieves higher prediction accuracy compared to advanced time series prediction models. Despite further improving the prediction accuracy, the existing LLMs neglect two crucial challenges for large-scale traffic flow prediction in 6G-IATS applications:

a) *Capturing of the spatio-temporal correlations in the large-scale road network*: Previous studies have shown that capturing the spatio-temporal correlations between roads in a large-scale road network can greatly improve the prediction

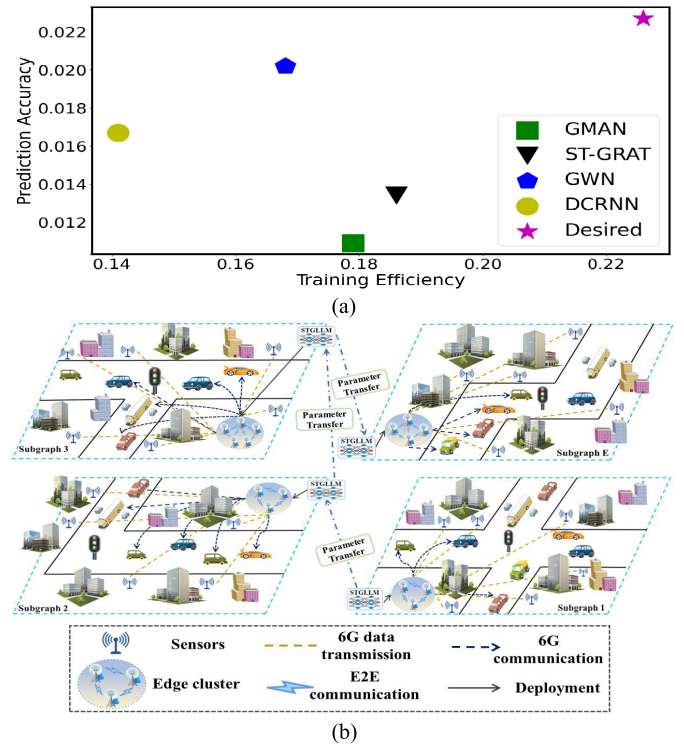


Fig. 1. (a) A motivation example. The evaluation is conducted on the LondonHW dataset with over 1000 roads. Training efficiency and prediction accuracy are assessed through computing the inverse of the total training time and the Mean Absolute Error (MAE), respectively. (b) The architecture of STGLLM-E for large-scale traffic flow prediction in 6G-IATS. A large-scale road network is segmented into E subgraphs. Each subgraph is covered by an edge cluster that can be flexibly deployed. The computing resources in edge clusters are determined by the computational load generated by subgraphs. Edge servers in a cluster can interact with each other through E2E communication, thus effectively achieving computation offloading. Sensors installed along the roads record traffic flow data, which is uploaded to their neighboring edge clusters driven by 6G. Meanwhile, we deploy an STGLLM on each edge cluster. An edge cluster is responsible for handling the traffic flow data, training the STGLLM, and predicting future traffic flow in the subgraph. Subsequently, future road traffic flow information is transmitted to nearby vehicles via 6G-assisted E2V communication for traffic control. Besides, to accelerate training, the parameter transfer strategy is used to initialize STGLLM deployed on different edge clusters.

accuracy of traffic flow [7], [8], [9]. Unfortunately, the LLM is a decoder-only structure, *i.e.*, it can only infer the future traffic flow according to historical traffic flow data, without the spatio-temporal features. Thus, it is essential to extract spatio-temporal correlations within the LLM.

b) *Improving the Training Efficiency*: To adapt the downstream tasks such as traffic flow prediction, LLMs are typically fine-tuned during training. Currently, most traffic flow prediction models are trained on a central server, which requires a large amount of training data to upload to this server [5], [6], [7], [8], [9]. If we adopt the training pattern to fine-tune an LLM on a large-scale road network, the following issue is caused. Since a large-scale road network produces more training data compared to the previous studies, the central server cannot meet the extremely high hardware requirements to store data and handle train tasks, leading to low efficiency. Moreover, considering challenge a), modeling the spatio-temporal correlations may bring more computational load to

the central server in the LLM, which leads to further degradation of training efficiency. Besides, uploading a large amount of training data to the central server may cause congestion, which also adversely affects training efficiency.

To sum up, two crucial challenges need to be considered in LLM-based large-scale traffic flow prediction in 6G-IATS: a) an LLM fails to extract the spatio-temporal correlations, and b) the training efficiency is low. Motivated by these challenges, we present a novel LLM and edge computing-based method entitled Spatio-Temporal Generative Large Language Model on Edge (STGLLM-E) for large-scale traffic flow prediction in 6G-IATS. In this architecture, the large-scale road network is represented as a graph. To reduce the scale of the graph, we segment the graph into several subgraphs. To capture the spatio-temporal correlations in each subgraph, an LLM-based method named Spatio-Temporal Generative Large Language Model (STGLLM) containing a Spatio-Temporal Module (STM) and a Generative Large Language Model (GLLM) is designed. To improve the training efficiency of STGLLM-E, we devise an edge training strategy based on edge servers. Particularly, the strategy first deploys an STGLLM on each edge cluster. An edge cluster contains several edge servers. Meanwhile, we assign traffic flow data and training tasks of each subgraph to corresponding edge cluster for training instead of uploading to a central server. Besides, a parameter transfer strategy is developed to initialize STGLLM deployed on different edge clusters. The idea of our proposed model is depicted in Fig. 1 (b). The contributions of our work are summarized as follows:

- We present a novel LLM and edge computing-based architecture STGLLM-E for large-scale traffic flow prediction in 6G-IATS. Specifically, the entire large-scale road network is treated as a graph. To reduce the scale of the graph, we design a method named RoadSort, which segments the graph into several subgraphs using Betweenness Centrality [13] and PageRank [14]. To capture the spatio-temporal correlations and infer in each subgraph, we propose an LLM-based method STGLLM with STM and GLLM. The former captures spatio-temporal correlations on such a small-scale subgraph. The latter with GPT-2 as the backbone aims to forecast future traffic flow. To the best of our knowledge, this paper constructs a brand-new hybrid architecture that integrates an LLM with a spatio-temporal model for large-scale traffic flow prediction in 6G-IATS.
- To improve the training efficiency of STGLLM-E, we develop an edge training strategy relying on edge servers. In detail, we first deploy an STGLLM on each edge cluster. The data and training tasks of each subgraph are allocated to the corresponding edge cluster rather than a central server for training. Furthermore, we design a parameter transfer strategy to initialize the STGLLM deployed on different edge clusters, which implements an “inference while train” mode to accelerate the training. In the parameter transfer, a pruning strategy is proposed to lighten the STGLLM, further enhancing the training efficiency.

- We conduct extensive experiments on two real-world large-scale traffic flow datasets, each comprising over 1000 roads. The experimental results indicate that STGLLM-E is significantly superior to advanced baselines in terms of prediction accuracy and training efficiency.

The rest of the paper is summarized as follows. Section II lists the related work on traffic flow prediction, LLMs on time series analysis, and edge computing. Section III presents the preliminaries and system overview of STGLLM-E. Section IV introduces the architecture design of STGLLM-E. In Section V, we introduce STGLLM-E’s training strategy. Section VI presents the experiments on two large-scale real-world traffic flow datasets. Section VII concludes this paper.

II. RELATED WORK

In this section, we review the related works, containing the previous methods for traffic flow prediction, LLMs on time series analysis, and edge computing.

A. Traffic Flow Prediction

Traffic flow prediction has been studied for decades. Classical statistical methods (e.g., ARIMA [5]) were initially adopted. However, these methods are unreliable for traffic flow prediction due to the feeble capability in mining the nonlinear features. After that, scholars presented the shallow machine learning methods (e.g., SVR [6]). Unfortunately, these methods cannot capture in-depth spatio-temporal correlations. This is primarily attributed to disjointed learning modules and the manual feature selection of these models, which cannot adapt to the massive traffic flow data from a road network.

With the rapid development of deep learning, numerous attempts have been undertaken to employ this technique in traffic flow prediction. In particular, Fu et al. applied the time series models like Long Short-Term Memory (LSTM) or Gated Recurrent Units (GRU) to predict short-term traffic flow [15]. However, they overlook the spatial dependencies between neighboring roads. In response, some works modeled a road network through a standard form (e.g., a two-dimensional (2D) matrix) and used a Convolutional Neural Network (CNN) to capture the spatial dependencies. Despite the benefits of convolution operation, some crucial information (e.g., the dependencies between distant roads) cannot be effectively represented in a 2D matrix, constraining the prediction precision. Considering this issue, GCN has received widespread attention as it utilizes the graph theory to treat the road network as a graph, which can adequately preserve the original spatial dependencies between roads.

Due to these merits, the various GCN-based models were presented to extract the spatio-temporal correlations in traffic flow data [7], [8]. For example, Li et al. proposed DCRNN, which substitutes linear operations in GRU with the diffusion convolution [16] and iteratively employs GCN to mine spatio-temporal correlations in each timestep [7]. Unfortunately, DCRNN is limited by predefined graph structures and the utilization of local spatio-temporal correlations. To overcome

this limitation, GWN was developed to capture global spatio-temporal features utilizing adaptive graphs, which effectively mitigates errors caused by predefined graph structures [8]. With the emergence of the Transformer [17], a MHSA mechanism with a powerful spatio-temporal feature extraction capability has been discovered. Specifically, GMAN parallelized spatial and temporal attention to capture spatial and temporal features, respectively [9]. A gating mechanism is then designed to adaptively fuse spatial and temporal features. In contrast to GCN-based models, Transformer-based models can further enhance the ability to mine global and dynamic spatio-temporal correlations. However, the large-scale road network has larger topological graph structures with more roads and intricate connectivity patterns, implying more complex spatio-temporal correlations [18]. Regardless of GCN or Transformer models, they demonstrate some challenges (e.g., poor prediction accuracy and low training efficiency) for large-scale traffic flow prediction in 6G-IATS, as shown in Fig. 1 (a).

B. LLMs on Time Series Analysis

As an emerging AI method, LLMs [10] have shown great strength on time series analysis tasks such as classification, imputation, and prediction [11], [12], [19]. For example, Zhou et al. proposed an LLM-based model named FPT to leverage a pre-trained GPT-2 with frozen MHSA layers and feedforward layers for various time series analysis tasks, containing imputation, classification, as well as long- and short-term prediction [11]. The study in [12] presented a time series prediction model entitled LLM4TS, which involves a two-stage training process: a) supervised pre-training using time series data, and b) fine-tuning the model according to specific tasks. In [19], GATGPT was developed to integrate GPT-2 and GAT for spatio-temporal interpolation. However, these LLM-based models fail to extract the spatio-temporal correlations in time series data. Similarly, for large-scale traffic flow prediction in 6G-IATS, the technique for effectively extracting spatio-temporal features of a large-scale road network within LLMs is not well-defined. Besides, the major scholars train their traffic flow prediction models on a central server, requiring all training data and computational tasks to be uploaded to this server. The large-scale road network generates more training data and computational tasks than the road network used in previous studies. If we take the training pattern to fine-tune an LLM using traffic flow data from the large-scale road network, it imposes a heavier load on the central server, leading to decreased training efficiency.

C. Edge Computing

Researchers have proposed many definitions related to edge computing. One definition state of edge computing is a technology that deploys edge devices to execute the computing tasks near the data source [20]. In [21], edge devices were classified into three categories: a) edge servers, which have weaker computing power than the traditional cloud computing devices (e.g., local cloud and Cloudlets [22]), b) coordination devices between terminal, which are less computing power and more

TABLE I
MAIN NOTATIONS

Notations	Explanation
G	The undirected graph generated by the large-scale road network
SG	The set of subgraphs derived from graph segmentation
$\ell_j(\cdot)$	The mapping function from historical traffic flow to prediction for n_j roads of the j th subgraph
H_j	The j th subgraph embeddings
HST^L	The output of L stacked ST blocks
HST^{l-1}	The input of the l th ST block
HS^l	The output of STBMHSA in the l th ST block
HST^l	The output of MHSA in the l th ST block
$HI_{v_i}^{Patch}$	The patched spatio-temporal representations of road v_i generated by tokenization in the j th subgraph
ev_i	The integration embedding of road v_i generated by patching encoding and position encoding in the j th subgraph
zv_i	The road v_i 's output embedding derived from pre-trained GPT-2
X_j	The past P timesteps traffic flow of the j th subgraph
Y_j	The prediction sequences of future Q timesteps for n_j roads in the j th subgraph

portable contrasted with edge servers, and c) device cloud, in which resource management and transmission between terminals are implemented via communication. In addition, the edge computing framework is composed of three tiers from bottom to top: a) mobile users, which serve as the data source of edge computing; b) edge servers, which are adopted to process and analyze data from mobile users; c) central servers, which provide centralized storage, integration, and coordination services. Within the framework, computing tasks from mobile users can be implemented on edge servers rather than central servers. Up till now, several edge computing-related applications have emerged in ITS, including the vehicle safety offloading [23] and traffic lights control [24]. Nonetheless, there are few studies on large-scale traffic flow prediction with edge computing in 6G-IATS, which is considered the motivation for our work.

III. PRELIMINARIES AND SYSTEM OVERVIEW

In this section, we introduce the basic definitions in this paper and formulate the problem. Then, the system overview of STGLLM-E is illustrated. For convenience, the notations are listed in Table I.

A. Definitions

Definition I. The Large-Scale Road Network: We define a large-scale road network as an undirected graph $G = (\mathbb{V}, \mathbb{E})$. In the graph, $\mathbb{V}$ is the set of nodes, containing $N = |\mathbb{V}|$ roads. A node represents a road. All the edges in the G are denoted as $\mathbb{E}$.

Definition II. Subgraphs: To reduce the scale of the large-scale road network, we segment the undirected graph G into some subgraphs. Since each edge cluster is responsible for a subgraph, the number of subgraphs is consistent with the number of edge clusters. Given E edge clusters in the G , we define the set of subgraphs as $SG = \{SG_1, SG_2, \dots, SG_E\}$, where the j th subgraph $SG_j = \{v_1, v_2, \dots, v_{n_j}\}$, $j \in [1, E]$. n_j stands for the total number of roads in the j th subgraph.

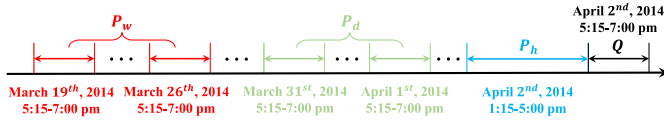


Fig. 2. An example of constructing the *recent*, *daily-periodic*, and *weekly-periodic* segment. Assuming that the sampling frequency c is 96 times per day, the size of the prediction window Q is 8 timesteps (2 hours). P_h , P_d , and P_w are twice of Q .

Definition III. Subgraph Representations: At timestep t , $X_j^t = \{x_{v_1}^t, x_{v_2}^t, \dots, x_{v_{n_j}}^t\} \in \mathbb{R}^{n_j \times F}$, $j \in [1, E]$ represents the traffic conditions of the j th subgraph, where $x_{v_i}^t$ indicates the traffic conditions of road v_i , $i \in [1, n_j]$. F denotes as the dimensions of traffic conditions, such as traffic flow and speed.

B. Problem Formulation

Based on the above definitions, the large-scale traffic flow prediction issue can be formally denoted as follows:

Problem. Large-scale Traffic Flow Prediction: Given an undirected graph G , we segment it into E subgraphs, contained within the set SG . For the j th subgraph, its subgraph representations are $X_j \in \mathbb{R}^{P \times n_j \times F}$ during the past P timesteps. Previous studies have demonstrated that traffic flow is influenced by recent hours, days, and weeks [25], [26]. Hence, the past P timesteps contain three segments: *recent*, *daily-periodic*, and *weekly-periodic*. Specifically, we assume that the sampling frequency is c times per day. Given the current timestep T and the size of predicting window Q , three time series segments of the length P_h , P_d , and P_w are intercepted along the time axis, serving as the inputs of *recent*, *daily-periodic*, and *weekly-periodic* segment, respectively. P_h , P_d , and P_w are integer multiples of Q . The three time series segments are described as follows:

1) The *recent*. As shown in the blue part of Fig. 2, the *recent* is a segment of the past time series directly adjacent to the prediction window, which can be denoted as $X_{j,h} = \{X_j^{T-P_h+1}, X_j^{T-P_h+2}, \dots, X_j^T\} \in \mathbb{R}^{P_h \times n_j \times F}$. Intuitively, traffic flow changes continuously. The just-past traffic flow impacts the traffic flow in the future.

2) The *daily-periodic*. As depicted in the green part of Fig. 2, the *daily-periodic* is composed of the segments on the past few days at the same time period as the prediction window, denoted as $X_{j,d} = \left\{ X_j^{T-\left(\frac{P_d}{Q}\right)*c+1}, \dots, X_j^{T-\left(\frac{P_d}{Q}\right)*c+Q}, X_j^{T-\left(\frac{P_d}{Q}-1\right)*c+1}, \dots, X_j^{T-\left(\frac{P_d}{Q}-1\right)*c+Q}, \dots, X_j^{T-c+1}, \dots, X_j^{T-c+Q} \right\} \in \mathbb{R}^{P_d \times n_j \times F}$. Since human daily life follows regular routine, traffic flow exhibits repetitive patterns, e.g., daily morning and evening rush hours. The *daily-periodic* segment enables the model to learn the daily periodicity of traffic flow.

3) The *weekly-periodic*. As illustrated in the red part of Fig. 2, the *weekly-periodic* is the segments on the few weeks at the same time interval as the prediction

window, represented as $X_{j,w} = \left\{ X_j^{T-7*\left(\frac{P_w}{Q}\right)*c+1}, \dots, X_j^{T-7*\left(\frac{P_w}{Q}\right)*c+Q}, X_j^{T-7*\left(\frac{P_w}{Q}-1\right)*c+1}, \dots, X_j^{T-7*\left(\frac{P_w}{Q}-1\right)*c+Q}, \dots, X_j^{T-7*c+1}, \dots, X_j^{T-7*c+Q} \right\} \in \mathbb{R}^{P_w \times n_j \times F}$. Traffic flow exhibits weekly periodicity. For instance, the variation of traffic flow on Monday is similar to that of historical Mondays but significantly different from that of weekends. Thus, the *weekly-periodic* is introduced to capture the weekly periodicity of traffic flow.

Then, the *recent*, *daily-periodic*, and *weekly-periodic* segment are concatenated to acquire the j th subgraph representations $X_j \in \mathbb{R}^{P \times n_j \times F}$:

$$X_j = [X_{j,h}, X_{j,d}, X_{j,w}], \quad (1)$$

where $[*, \dots, *]$ is the concatenation operation. P equals the sum of P_h , P_d , and P_w .

In this paper, we intend to learn a mapping function $\ell_j(SG_j, X_j)$ to forecast traffic flow of n_j roads in the future Q timesteps:

$$Y_j = \ell_j(SG_j, [X_{j,h}, X_{j,d}, X_{j,w}]), \quad (2)$$

where $Y_j = \{Y_j^{T+1}, Y_j^{T+2}, \dots, Y_j^{T+Q}\} \in \mathbb{R}^{Q \times n_j \times F}$ is the predicted output. By the iterative training, the optimal mapping ℓ to each subgraph is learned. When each subgraph completes its prediction, we can acquire the prediction results on the G . Note that since we focus on forecasting traffic flow, F is set to 1.

C. System Overview

As depicted in Fig. 3, to address on the above challenges, we present the STGLLM-E.

First, an undirected graph is constructed to represent the large-scale road network. To reduce the scale, a novel graph segmentation technique named RoadSort is designed, which segments the graph into several subgraphs. Specifically, RoadSort follows the basic principles of PageRank [14], the most commonly used graph node importance ranking algorithm. Moreover, we introduce the Betweenness Centrality [13] to incorporate the number of the shortest paths through the roads into PageRank. Compared with the traditional PageRank, RoadSort can more accurately identify the important roads and retain the edges of these roads, reducing the loss of important information.

Second, for each subgraph, a Fully Connected (FC) layer is adopted to reshape the subgraph representations. The transformed subgraph representations are then fed to an LLM-based method STGLLM, which incorporates STM and GLLM. The former is adopted to extract the spatio-temporal correlations. The latter is leveraged to infer prediction sequences. Specifically, STM is composed of L Spatio-Temporal (ST) blocks. In each ST block, since MHSA is effective in dynamically adjusting the degree of model attention in both the temporal and spatial dimensions, enabling us to effectively handle the complex relationships of these two dimensions, we factorize the spatio-temporal modeling along spatial and temporal

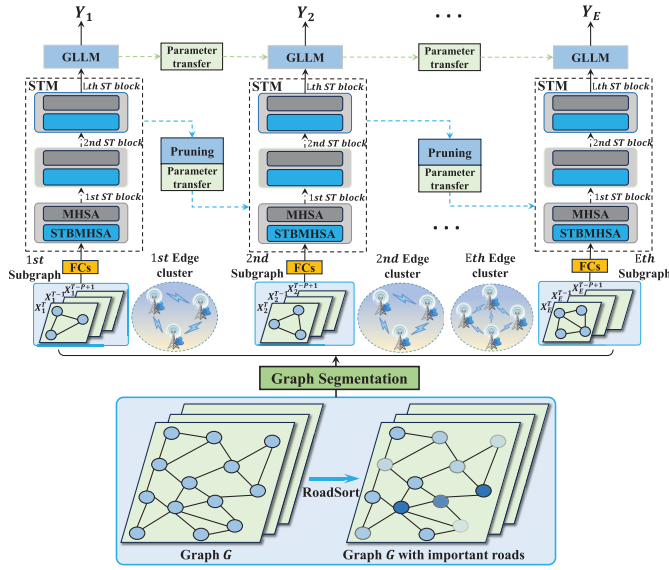


Fig. 3. The overview of STGLLM-E.

domains, forming the dual levels of MHSA: a) Square Target Board Multi-Head Self Attention (STBMHSA) for learning the spatial dependencies, and b) the standard MHSA for extracting temporal dependencies in each road. After receiving the spatio-temporal correlations from STM, GLLM with GPT-2 predicts the future traffic flow. Because the pre-trained LLMs with billions of corpora can bring the extensive intrinsic knowledge to generate the prediction sequences.

Last but not least, we propose an edge training strategy based on edge computing to improve the training efficiency of STGLLM-E, since edge computing offloads computing tasks from a central server to edge servers near to IoT devices, enormously reducing the amount of data transmitted to the central server. In detail, we deploy a separate STGLLM on each edge cluster. Each subgraph representations are uploaded to the corresponding edge cluster for training. Besides, a parameter transfer strategy is applied to initialize the STGLLM deployed on different edge clusters, accelerating the training. In the transfer process of STM, a pruning strategy is adopted to reduce the headcount of MHSA and STBMHSA transferred to the next edge cluster, aiming to design the lightweight STGLLM.

IV. STGLLM-E DETAILED ARCHITECTURE

In this section, the architecture design of STGLLM-E is introduced. In particular, we first illustrate the graph segmentation method RoadSort. Then, we demonstrate the description of STGLLM.

A. Graph Segmentation

As mentioned in the above content, we segment the entire graph into several subgraphs to reduce the scale of the large-scale road network. Unfortunately, many edges connecting to roads in the graph are cut off during the segmentation, resulting in the loss of crucial information. It can adversely affect training and prediction. Hence, reducing information loss is essential to graph segmentation.

The road importance $\mathcal{R}$ is an indispensable factor in the large-scale road network, which is beneficial to traffic planning and analysis [27]. If the important roads are identified, we can treat them as the center roads to construct the subgraphs with the neighboring roads around them. In this way, the edges of the central roads with abundant information are preserved. For this purpose, it is necessary to design an effective method to measure the road importance $\mathcal{R}$. In [28], three indicators are adopted to measure $\mathcal{R}$, including the number of edges connected to the road, the weights of edges connected to the road, and the significance of the other roads connected to the road. Jointly considering these three indicators, a RoadSort method is designed to calculate each road's importance in the large-scale road network, assisted by combining PageRank with Betweenness Centrality. Next, we describe the design in detail.

1) *Betweenness Centrality*: Ulrik and Brandes [13] define Betweenness Centrality as the sum of the shortest paths passing through the road, formulated as:

$$B_c(v_R) = \frac{\sum_{S=1}^N \sum_{T=1}^N \rho_{v_S, v_T}(v_R)}{\sum_{S=1}^N \sum_{T=1}^N \rho_{v_S, v_T}}, \quad s.t. \ v_S \neq v_R, v_T \neq v_R, v_T \neq v_S, \quad (3)$$

where $\rho_{v_S, v_T}(v_R)$ stands for the shortest path from road v_S to road v_T that passes through road v_R . The road v_S , v_T , and v_R belong to the set $\mathbb{V}$. The sum of the shortest paths from v_S to v_T is given by ρ_{v_S, v_T} . According to the above definition, we can find a road with a larger B_c is more likely to become a transportation hub since it is the shortest path for many routes in the large-scale road network.

2) *PageRank*: In [14], Page et al. proposes the PageRank algorithm to sort web pages according to their importance. PageRank can be defined on any graph, including a large-scale road network. Specifically, given that the degree of road v_S is d , the probability of vehicles moving from road v_S , $S \in [1, N]$ to road v_T , $T \in [1, N]$ is defined as

$$\mathcal{P}_{v_S, v_T} = \begin{cases} 1/d, & v_S \text{ links to } v_T \\ 0, & \text{otherwise,} \end{cases} \quad (4)$$

where $\mathcal{P}_{v_S, v_T}$ stands for the transition probability from road v_S to road v_T , and $\mathcal{P} = [\mathcal{P}_{v_S, v_T}]_{N \times N} \in \mathbb{R}^{N \times N}$ denotes as the transition matrix. The transition matrix exists two properties: 1) $\mathcal{P}_{v_S, v_T} \geq 0$, $\forall S, T \in [1, N]$, and 2) $\sum_{S=1}^N \mathcal{P}_{v_S, v_T} = 1$.

Let the road importance of N roads in the graph G be $\mathcal{R} = [PR(v_1), PR(v_2), \dots, PR(v_N)]^T$, where $PR(v_N)$ is the PageRank value of road v_N . Assuming a complete random walk model, each element in $\mathcal{P}'$ is $1/N$. The general form of PageRank is

$$\begin{aligned} \mathcal{R} &= \beta \cdot \mathcal{P} \cdot \mathcal{R} + (1 - \beta) \cdot \mathcal{P}' \\ &= \beta \cdot \mathcal{P} \cdot \mathcal{R} + \frac{1 - \beta}{N}, \end{aligned} \quad (5)$$

where $\beta \in [0, 1]$ is the damping factor, representing the resistance from one road to another.

3) *RoadSort*: However, if the PageRank algorithm is directly used to the large-scale road network, the following issues arise: a) PageRank fails to identify some important

roads, such as the transportation hubs, and b) the distance between roads is a key factor that determines the information diffusion and propagation. While we define the transition matrix in PageRank, the distance factor is neglected. In view of this, a RoadSort method is designed to adapt the characteristics of the large-scale road network and assess road importance more realistically. In detail, those roads potentially serving as transportation hubs are more crucial. To identify transportation hubs, Betweenness Centrality is first adopted to compute the weights of the roads. Redefine $\mathcal{P}$ as:

$$\mathcal{P}_{v_S, v_T} = \begin{cases} \frac{B_C(v_S)}{\sum_{S=1}^N B_C(v_S)} \cdot \frac{B_C(v_T)}{\sum_{T=1}^N B_C(v_T)}, & v_S \text{ links to } v_T \\ 0, & \text{otherwise,} \end{cases} \quad (6)$$

where $B_C(v_S)$ and $B_C(v_T)$ denote the Betweenness Centrality of the road v_S and v_T , respectively. $\sum_{S=1}^N B_C(v_S)$ and $\sum_{T=1}^N B_C(v_T)$ stand for the sum of Betweenness Centrality values of the roads connected with v_S and v_T , respectively. The redefined transition matrix $\mathcal{P}$ has two properties: 1) $\mathcal{P}_{v_S, v_T} \geq 0$, $\forall S, T \in [1, N]$, and 2) $\sum_{S=1}^N \sum_{T=1}^N \mathcal{P}_{v_S, v_T} = 1$.

The damping factor reflects the resistance from one road to another. Studies show that the distance between roads is a strong resistance to traveling [29]. But in PageRank, the damping factor is set to a constant value. Considering the problem, we adopt the distance information to compute the factor. Let β be a diagonal matrix $\beta = (\beta_{v_1}, \beta_{v_2}, \dots, \beta_{v_N})$, in which β_{v_T} is denoted as:

$$\beta_{v_T} = \gamma \cdot \frac{1}{\sum_{v_S} \frac{1}{d_{v_S, v_T}}}, \quad (7)$$

where d_{v_S, v_T} is the distance between road v_S and road v_T . In the undirected graph G , a road can be represented as a road segment or a crossroad. Hence, the roads are not necessarily connected in a straight line. The distance between road v_S and road v_T is not the Euclidean distance, but the shortest distance that can be reached to each other along the road. γ stands for a scaling factor. Then, we reconstruct the PageRank as:

$$\mathcal{R} = \begin{bmatrix} \beta_{v_1} & & \\ & \ddots & \\ & & \beta_{v_N} \end{bmatrix} \cdot \mathcal{P} \cdot \mathcal{R} + \frac{1}{N} \begin{bmatrix} 1 - \beta_{v_1} & & \\ & \ddots & \\ & & 1 - \beta_{v_N} \end{bmatrix}. \quad (8)$$

Through the road importance $\mathcal{R}$, we select the top- E roads as the center roads to segment the G and construct E subgraphs, which can be denoted as $SG = \{SG_1, SG_2, \dots, SG_E\}$. In this way, we retain the edges of the crucial roads, which contain more valuable information. It enhances the prediction accuracy.

B. STGLLM

For each subgraph, to capture the spatio-temporal correlations and infer the future traffic flow, we develop the STGLLM

method. As depicted in Fig. 4 (a), STGLLM is composed of three components, containing Subgraph Embedding Layer, STM, and GLLM. The three components take a pipeline architecture. Next, we discuss the details of each component and how they collaborate to capture the spatio-temporal correlations and predict the future traffic flow, taking the processing of the j th subgraph as an example.

1) *Subgraph Embedding Layer*: Since the neural networks cannot directly handle the raw traffic flow data, it is necessary to convert the j th subgraph representations X_j into a higher-dimensional space. Particularly, an FC with two layers is employed to convert the dimensions from F to d_{model} , which can be denoted as $H_j = \{H_j^{T-P+1}, H_j^{T-P+2}, \dots, H_j^T\} \in \mathbb{R}^{P \times n_j \times d_{\text{model}}}$. d_{model} is the dimensions of the hyperparameter of STGLLM.

2) *STM*: As mentioned in the previous content, the traffic flows in the j th subgraph are affected by the several factors. Specifically, in the spatial dimension, the spatial dependencies influence the variation of traffic flow. For instance, a large influx of vehicles from neighboring roads brings about the congestion and a dramatic decrease in the traffic flow on the target road. In the temporal dimension, the traffic flows on each road are affected by the temporal dependencies. For example, the congestion during morning rush hour is impacted by the previous traffic rhythm. The influence gradually increases over time until alleviating. Hence, it is essential to extract the spatio-temporal correlations in the j th subgraph embeddings.

Up till now, we can acquire the j th subgraph embeddings of past P timesteps $H_j \in \mathbb{R}^{P \times n_j \times d_{\text{model}}}$ generated by the previous subgraph embedding layer. Then, STM is presented to capture the spatio-temporal correlations from these embedded features. As shown in Fig. 4, STM consists of L stacked ST blocks. Given $H_j \in \mathbb{R}^{P \times n_j \times d_{\text{model}}}$ as the inputs of L ST blocks, the final spatio-temporal representations generated by the L th ST block are $HST^L \in \mathbb{R}^{P \times n_j \times d_{\text{model}}}$. Each ST block is composed of two structures, containing STBMHSA and MHSA. These two structures take a pipeline form. The former is first designed to extract the spatial dependencies. The latter is then developed to capture the temporal dependencies.

STBMHSA: The previous studies have adopted the standard MHSA to extract the spatial dependencies [9]. Such a model focuses on the spatial relationships between the target road and all other roads in a road network. In this way, given the j th subgraph embeddings of n_j roads at timestep t $H_j^t \in \mathbb{R}^{n_j \times d_{\text{model}}}$, the computational complexity of the standard MHSA for capturing the spatial dependencies is $\mathcal{O}(n_j^2 d_{\text{model}})$. This quadratic cost increases the computational complexity of MHSA, leading to inefficient training. To address the issue, we design an improved version of MHSA entitled STBMHSA, which efficiently extracts the spatial dependencies within each timestep. Specifically, considering that neighboring roads with close space distribution have similar spatial dependencies to the target road, and the spatial dependencies between nearby roads are always stronger than those far away, each road attends to its surroundings in a square target board fashion, *i.e.*, close regions at a fine granularity and distant regions at a coarse granularity. Hence,

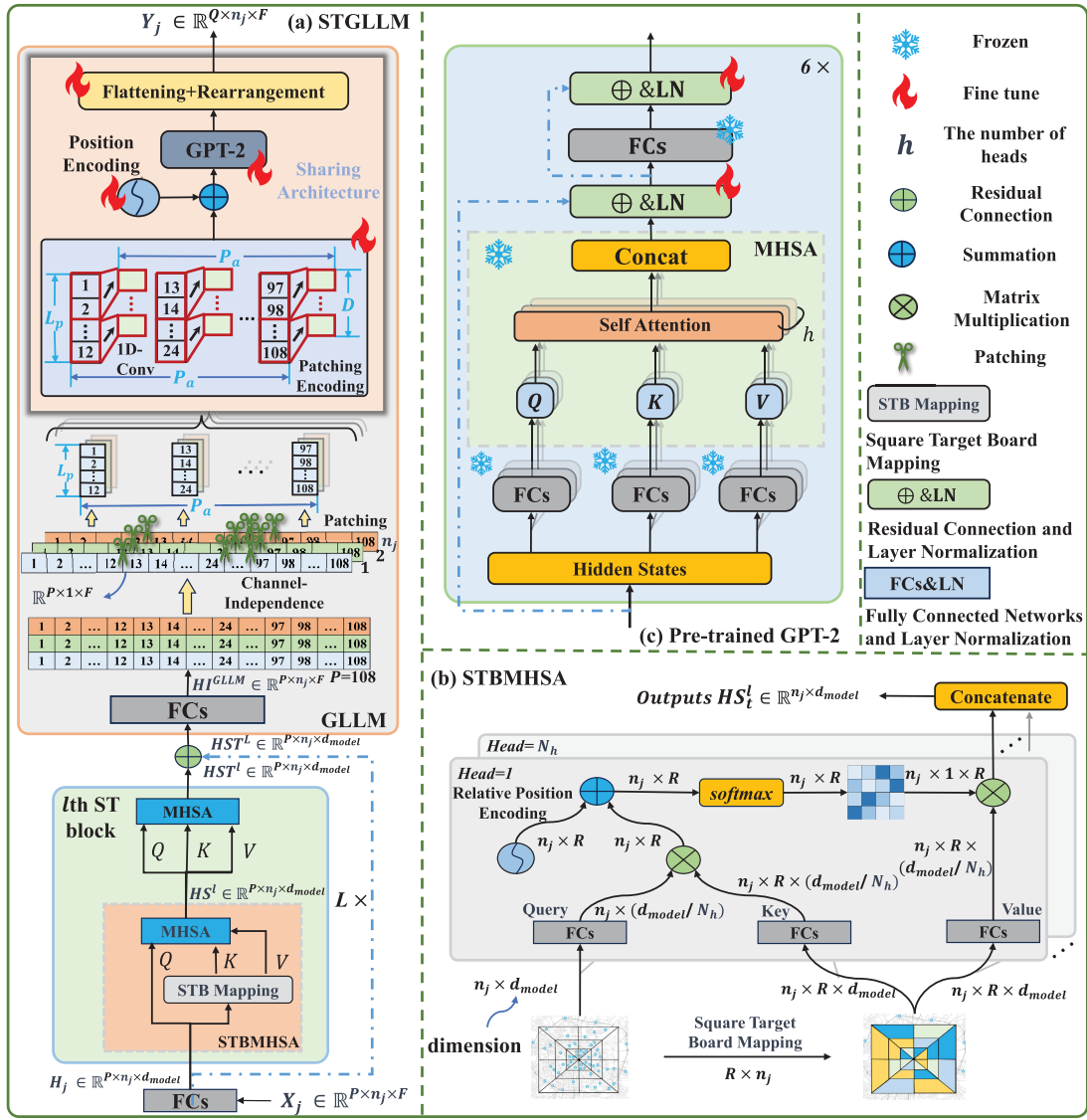


Fig. 4. The framework of STGLLM. **a) STGLLM.** For the j th subgraph, the subgraph representations X_j is the input of subgraph embedding layer. H_j is the input of L stacked ST blocks. In the l th ST block, HS^l stands for the output of STBMHSA. HST^l is the output of l th ST block. The final output of L stacked ST blocks is HST^L . HJ^{GLLM} is the input of tokenization. $\mathbb{R}^{P \times 1 \times F}$ is the dimensions of the restored spatio-temporal representations of an individual road generated by channel-independence. P_a is the number of patches. L_p is the length of each patch. Y_j is the output of STGLLM. Note that we assume $P = 108$ and $L_p = 12$ in this example. In GLLM, multi-road spatio-temporal representations are divided into different channels. These channels share the same patching encoding and position encoding, GPT-2, and output layer, but the tokenization is independent. **b) STBMHSA.** In the l th ST block, HST^{l-1} is the input of STBMHSA. HS^l is the output of STBMHSA. N_h is the headcount of self attention. **c) pre-trained GPT-2.**

as shown in Fig. 5 (a), we divide the j th subgraph into the several regions in the form of a square target board, which is centered on the target road. MHSA is then used to compute the spatial dependencies between each region and the target road, shown in Fig. 5 (b) and (c).

In the l th ST block, we define the spatio-temporal representations generated by the $l - 1$ th ST block $HST^{l-1} \in \mathbb{R}^{P \times n_j \times d_{model}}$ as the input of STBMHSA, in which the representations at timestep t are $HST_t^{l-1} \in \mathbb{R}^{n_j \times d_{model}}$. The spatial representations generated by STBMHSA are $HS^l \in \mathbb{R}^{P \times n_j \times d_{model}}$, in which the representations are $HS_t^l \in \mathbb{R}^{n_j \times d_{model}}$ at timestep t .

Square Target Board Mapping & MHSA: In detail, for road v_i at timestep t , the formulation of the square target board mapping is unified by introducing a mapping matrix

$A_{v_i} \in \mathbb{R}^{R \times n_j}$, which represents how nearby roads in the j th subgraph are projected to R regions bounded by line segments and rectangles. Each element $a_{r,v_i} \geq 0$ in the mapping matrix A_{v_i} is the likelihood that road v_i falls into the region r . A_{v_i} satisfies that the sum of the elements in each row is 1.

Fig. 5 illustrates an example of how we segment the surroundings of a target road in a square target board fashion. The farthest ring is farthest from the target road, while the innermost ring is the nearest. The three rings focus on the target road as their common center and are further partitioned by four line segments into eight different directions. The roads locating outside the farthest ring are not within the reference scope. In this case, the value of R is $24 + 1 = 25$, in which $+1$ indicates the target road itself.

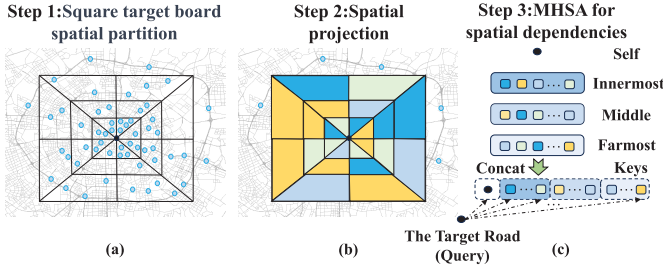


Fig. 5. Schema of STBMHSA for the j th subgraph. For a target road (a black spot), its surroundings are segmented into several regions bounded by three squares and four line segments. Furthermore, neighboring roads are mapped into these regions to acquire regional representations. Ultimately, we treat the target road as the query and the regional representations as keys and values for multi-head self attention operation.

Let $HST_t^{l-1} \in \mathbb{R}^{n_j \times d_{\text{model}}}$ be the input of square target board mapping. The input features of n_j roads are mapped to regional representations that correspond to each road via the mapping matrix A_{v_i} :

$$C_{v_i} = A_{v_i} HST_t^{l-1}, C = [C_{v_1}, C_{v_2}, \dots, C_{v_{n_j}}], \quad (9)$$

where $C_{v_i} \in \mathbb{R}^{R \times d_{\text{model}}}$ is the regional representations of road v_i as a target road. $C \in \mathbb{R}^{n_j \times R \times d_{\text{model}}}$ stands for the regional representations of n_j roads after mapping.

Furthermore, we perform MHSA to extract the spatial dependencies between each target road and its neighboring regions. Specifically, the h th head operation of the MHSA is as follows:

$$HST_{h,t}^l = \text{Softmax} \left(\alpha Q_h^S (K_h^S)^\top + B_h \right) V_h^S, \quad (10)$$

where $HST_{h,t}^l \in \mathbb{R}^{n_j \times \left(\frac{d_{\text{model}}}{N_h}\right)}$ is the spatial representations of the h th head at timestep t . The query $Q_h^S \in \mathbb{R}^{n_j \times \left(\frac{d_{\text{model}}}{N_h}\right)}$, key $K_h^S \in \mathbb{R}^{n_j \times R \times \left(\frac{d_{\text{model}}}{N_h}\right)}$, and value $V_h^S \in \mathbb{R}^{n_j \times R \times \left(\frac{d_{\text{model}}}{N_h}\right)}$ are acquired by linear mapping $HST_t^{l-1} W_Q^S$, $C W_K^S$, and $C W_V^S$, respectively. W_Q^S , W_K^S , and $W_V^S \in \mathbb{R}^{d_{\text{model}} \times \left(\frac{d_{\text{model}}}{N_h}\right)}$ are the learnable parameters for linear mapping. N_h is the headcount. $B_h \in \mathbb{R}^{n_j \times R}$ is relative position encoding that takes position information into account. α is a scaling factor. Fig. 4 (b) shows the better understanding the process.

After acquiring $HST_{h,t}^l$, and $h \in [1, N_h]$, the spatial representations $HS_t^l \in \mathbb{R}^{n_j \times d_{\text{model}}}$ at timestep t are formulated as:

$$HS_t^l = [HST_{1,t}^l, HST_{2,t}^l, \dots, HST_{N_h,t}^l] W_o^S, \quad (11)$$

where $W_o^S \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$ denotes the learnable parameters.

Discussion: Due to the small number of regions ($R \ll n_j$), the computational complexity of STBMHSA $\mathcal{O}(R n_j d_{\text{model}})$ increases linearly with the growth of the number of roads in the j th subgraph, which is more efficient than the standard MHSA. In addition, By employing square target board mapping, the STBMHSA doesn't introduce the additional parameters to the standard MHSA, ensuring lightweight in reality.

MHSA: To capture temporal dependencies, we propose the MHSA. By an attention mechanism, this structure can

dynamically assign weights to different timesteps according to their importance.

In the l th ST block, let the spatial representations of past P timesteps $HS^l \in \mathbb{R}^{P \times n_j \times d_{\text{model}}}$ be input of MHSA, in which $HS_{v_i}^l$ denotes as the spatial representations of road v_i . The spatio-temporal representations generated by this structure are represented as $HST^l \in \mathbb{R}^{P \times n_j \times d_{\text{model}}}$, in which $HST_{v_i}^l \in \mathbb{R}^{P \times d_{\text{model}}}$ is the spatio-temporal representations of road v_i . In the end, we acquire the spatio-temporal representations of the l th ST block through residual connection.

For road v_i , the operation of the h th head is expressed as:

$$HST_{h,v_i}^l = \text{Softmax} \left(\alpha Q_h (K_h)^\top \right) V_h, \quad (12)$$

where $HST_{h,v_i}^l \in \mathbb{R}^{P \times \left(\frac{d_{\text{model}}}{N_h}\right)}$ is the temporal representations of the h th head. $Q_h = HS_{v_i}^l W_Q$, $K_h = HS_{v_i}^l W_K$, and $V_h = HS_{v_i}^l W_V$ are query, key, and value, respectively. W_Q , W_K , and $W_V \in \mathbb{R}^{d_{\text{model}} \times \left(\frac{d_{\text{model}}}{N_h}\right)}$ are the learnable parameters for the linear mapping.

Once HST_{h,v_i}^l , $h \in [1, N_h]$ is acquired, the spatio-temporal representations $HST_{v_i}^l \in \mathbb{R}^{P \times d_{\text{model}}}$ of road v_i are expressed as:

$$HST_{v_i}^l = [HST_{1,v_i}^l, HST_{2,v_i}^l, \dots, HST_{N_h,v_i}^l] W_o, \quad (13)$$

where $W_o \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$ denotes the learnable parameters.

3) **GLLM:** After receiving the spatio-temporal features from STM, we predict the future traffic flow using these features. In light of this, we propose the LLM-based method GLLM, which consists of four structures, namely tokenization, patching encoding and position encoding, GPT-2 [10], and output layer. Next, we describe how each structure is applied and collaborates with each other to infer future traffic flow.

Tokenization: Traffic flow prediction aims to learn the dependencies between data in each different timestep. Unfortunately, a single timestep does not have semantic meaning like a word in a sentence. Thus, capturing local semantic information is vital in analyzing their connections. In view of this, we employ channel-independence along with patching [30] for the spatio-temporal representation tokenization. As shown in Fig. 4 (a), since GPT-2 cannot directly deal with the high-dimensional spatio-temporal features $HST^L \in \mathbb{R}^{P \times n_j \times d_{\text{model}}}$, an FC with two layers is first adopted to restore the dimensions from d_{model} to F , denoted as $HI^{GLLM} \in \mathbb{R}^{P \times n_j \times F}$. Channel-independence then converts the multi-road spatio-temporal representations into spatio-temporal representations of multiple independent roads. A road is considered a channel, thus transforming the representations' dimension to $\mathbb{R}^{P \times 1 \times F}$. Finally, for the spatio-temporal representations of each road, patching is proposed to group adjacent timesteps into a singular patch-based token, reducing the dimensions from P to P_a , where P_a stands for the total number of patches. Meanwhile, we expand the feature dimensions from $1 \times F$ to $L_p \times F$, where L_p denotes the length of each patch. Particularly, given the restored spatio-temporal representations HI^{GLLM} , we adopt channel-independence (CI) and patching

to generate a sequence of patches:

$$HI_{v_i}^{Patch} = \text{patching} \left(\text{CI} \left(HI^{GLLM} \right) \right), i \in [1, n_j], \quad (14)$$

where $HI_{v_i}^{Patch} \in \mathbb{R}^{P_a \times L_p \times F}$ is the patched spatio-temporal representations of road v_i .

Discussion: Channel-independence avoids complex cross-channel operations and allows each channel to be processed in parallel, thus reducing the demand for computational resources. The design of patching naturally has two-fold benefit: a) patching can retain local semantic information through aggregating timesteps into subseries-level patches, and b) by using patching, the number of patches reduces from P to P_a . It indicates that the computational complexity and memory usage of the self attention mapping in the GPT-2 are quadratically reduced by a factor of $P - P_a$. Thus restricted on the GPU memory and training time, patching facilitates the model to see longer historical sequences, which requires more valuable information.

Patching Encoding and Position Encoding: In NLP practices, the token encoding is proposed to align the dimensions of patches with the embedding dimensions of GPT-2. And in our work, it is also necessary to utilize the token encoding to process the patched spatio-temporal representations, ensuring that their dimensions are consistent with the embedding dimensions of GPT-2. However, the token encoding is specifically developed to handle text sequences in GPT-2. Such a method is only suitable for scalar tokens, while the patched spatio-temporal representations are vectors. To address this problem, we develop a novel token encoding named patching encoding. In detail, we adopt a 1D convolutional layer **Conv** as our patching encoding layer. Compared with a linear layer [11] in the patching encoding layer, we select the 1D convolutional layer because of its outstanding capacity to preserve the local semantic information within the patched spatio-temporal representations. Hence, given the road v_i 's patched spatio-temporal representations $HI_{v_i}^{Patch} \in \mathbb{R}^{P_a \times L_p \times F}$, the patching encoding based on the 1D convolutional layer is as follows:

$$e_{v_i}^{\text{token}} = \text{Conv} \left(HI_{v_i}^{Patch} \right), \quad (15)$$

where $e_{v_i}^{\text{token}} \in \mathbb{R}^{P_a \times D \times F}$ is the patching embedding generated by the patching encoding. D denotes the dimensions of patching embedding, consistent with embedding dimensions of GPT-2.

To learn the order and position relationships between the different timesteps in the patched spatio-temporal representations, we introduce the position encoding inspired by Transformer [17]. Given the index of the road v_i 's patch locations $ind_{v_i} \in \mathbb{R}^{P_a}$, we adopt a learnable lookup table $\mathbf{E}^{\text{pos}}$ to project the patch locations:

$$e_{v_i}^{\text{pos}} = \mathbf{E}^{\text{pos}} (ind_{v_i}), \quad (16)$$

where $e_{v_i}^{\text{pos}} \in \mathbb{R}^{P_a \times D \times F}$ denotes the position embedding generated by the position encoding.

After obtaining the patching embedding $e_{v_i}^{\text{token}}$ and the position embedding $e_{v_i}^{\text{pos}}$, we sum them to yield the integration embedding $e_{v_i} \in \mathbb{R}^{P_a \times D \times F}$:

$$e_{v_i} = e_{v_i}^{\text{token}} + e_{v_i}^{\text{pos}}. \quad (17)$$

e_{v_i} is then fed into the GPT-2.

Discussion: As mentioned in the previous content, patching intends to aggregate neighboring timesteps to construct a singular patch-based token, thus preserving local semantic information. Different from patching, patching encoding is responsible for projecting the output of patching to the required dimensions of the subsequent GPT-2.

GPT-2: To retain LLMs' ability in data-independent representation learning, most parameters in LLMs are frozen. Some studies prove that training LLMs from scratch impairs performance, confirming the significance of freezing most parameters to maintain the ability in representation learning [10], [11]. In view of this, the pre-trained GPT-2 is applied to generate output embedding. Its structure is shown in Fig. 4 (c). Specifically, we choose to freeze FC layers and MHSA, as they have a significant impact on the representation learning [11].

For the rest trainable parameters (e.g., layer normalization) in the pre-trained GPT-2, we adopt Layer Normalization Tuning strategy [31] to fine-tune them, enabling the affine transformation of layer normalization to be trained. Within the pre-trained GPT-2, only 1.5% of the total parameters need to be fine-tuned.

Given the integration embedding $e_{v_i} \in \mathbb{R}^{P_a \times D \times F}$, we feed it into the GPT-2 with the six pre-trained Transformer blocks (**Tbs**). The process produces the road v_i 's output embedding $z_{v_i} \in \mathbb{R}^{P_a \times D \times F}$:

$$z_{v_i} = \mathbf{Tbs}(e_{v_i}). \quad (18)$$

Output Layer: After receiving the output of the pre-trained GPT-2, we transform the output embedding $z_{v_i} \in \mathbb{R}^{P_a \times D \times F}$ to the output sequences by output layer with flattening and rearrangement. Specifically, we first flatten z_{v_i} into a 1D vector. Rearrangement is then used to acquire the road v_i 's traffic flow $y_{v_i} \in \mathbb{R}^{Q \times 1 \times F}$ in the future Q timesteps, and $y_{v_i} \in Y_j$.

$$y_{v_i} = \text{Rearrange} \left(\text{Flatten} (z_{v_i}) W_f^T \right), \quad (19)$$

where $W_f^T \in \mathbb{R}^{Q \times (P_a D F)}$ stands for the learnable weight. For n_j roads in the j th subgraph, we first adopt tokenization to independently process these roads to acquire the patched spatio-temporal representations of each road. Then, the patched spatio-temporal representations of n_j independent roads share the remaining GLLM architecture (e.g., patching encoding and position encoding, GPT-2, and output layer) to output n_j roads' traffic flow $Y_j \in \mathbb{R}^{Q \times n_j \times F}$ in the future Q timesteps.

V. TRAINING STRATEGY OF STGLLM-E

In this section, we introduce the training strategy of STGLLM-E.

Since E subgraphs generate a large amount of training data, uploading these data to a central server for the model training brings about the significant computational load and transmission delay, leading to inefficient training. To address this problem, we propose an edge training strategy. Given E edge clusters that can be flexibly deployed on E subgraphs, each cluster contains several edge servers with identical computing power. The amount of edge servers in a cluster indicates

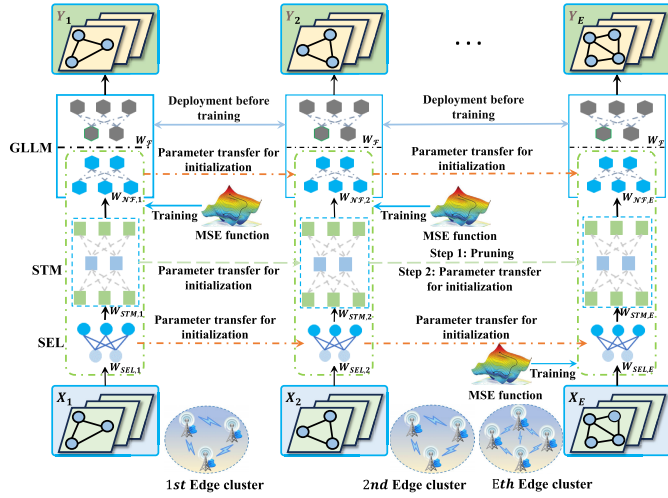


Fig. 6. Edge training strategy. SEL is the subgraph embedding layer.

the computing resources assigned to a subgraph, which can be determined by the number of roads in the subgraph. Specifically, different subgraphs contain varying numbers of roads. When more roads are included in a subgraph, data processing requests will also increase. So the computing resources should be adjusted accordingly, *i.e.*, more edge servers are needed in a cluster. The computing resources allocated to the j th subgraph are calculated by

$$CR_j = \left\lfloor \frac{n_j}{N} \right\rfloor CR_{\text{sum}}, \quad (20)$$

where $\lfloor \cdot \rfloor$ is the floor symbol. CR_{sum} represents the sum of computing resources. CR_j is the number of edge servers in the j th edge cluster.

As shown in Fig. 6, we define $W_j = \{W_{\text{SEL},j}, W_{\text{STM},j}, W_{\text{GLLM},j}\}$, $j \in [1, E]$ as the parameters of the j th STGLLM, where $W_{\text{SEL},j}$ is the parameters of the subgraph embedding layer. $W_{\text{STM},j}$ stands for the parameters of STM. $W_{\text{GLLM},j} = \{W_{\mathcal{F}}, W_{\mathcal{NF},j}\}$ is the parameters of GLLM. $W_{\mathcal{F}}$ represents the frozen parameters in GLLM, including the MHSA layers and FCs in the pretrained GPT-2, while $W_{\mathcal{NF},j}$ denotes the remaining unfrozen parameters in GLLM. Before the training, the j th subgraph representations X_j are uploaded to the j th edge cluster. Since $W_{\mathcal{F}}$ does not participate in training, we uniformly deploy these frozen model structures to each edge cluster.

Furthermore, due to the continuity of roads between the adjacent regions, we generally believe that the variations of traffic flow between the neighboring subgraphs exhibit similarities [32]. For instance, during morning and evening rush hours in the Central Business District (CBD), traffic congestion not only affects the CBD but also spreads to the surrounding regions. As a result, transfer learning is introduced to initialize the parameters of STGLLM deployed on the neighboring edge cluster. We first train the 1st STGLLM using X_1 on the 1st edge cluster. Specifically, we optimize the parameters of STGLLM for the n_1 roads in the 1st subgraph by minimizing Mean Squared Error (MSE) loss function, denoted as:

$$\text{MSE} = \frac{\sum_{m=1}^Q \sum_{v_i=1}^{n_1} (\hat{x}_{v_i}^m - y_{v_i}^m)^2}{Q \times n_1} + \frac{\lambda}{2} \|W_1^{\text{train}}\|^2, \quad (21)$$

where Q denotes the length of the predicted timesteps. The observed and predicted values at the timestep m on the road v_i are $\hat{x}_{v_i}^m \in \hat{X}_1$ and $y_{v_i}^m \in Y_1$, respectively. $\hat{X}_1 \in \mathbb{R}^{Q \times n_1 \times F}$ is the traffic flow observations of n_1 roads at future Q timesteps. λ stands for the regularization. $W_1^{\text{train}} = \{W_{\text{SEL},1}, W_{\text{STM},1}, W_{\mathcal{NF},1}\}$ is the learnable parameter. After the training, the STGLLM produces output sequences on the edge cluster. Meanwhile, W_1^{train} is transferred to the 2nd edge cluster adjacent to the 1st edge cluster for the initialization. Neighboring subgraph representations X_2 are then adopted to train based on the initialization. After the multiple rounds of iteration, we can obtain the optimized W_2^{train} . Similarly, W_2^{train} not only generates the prediction sequences of n_2 roads on the 2nd edge cluster, but also transmits to the next neighboring edge cluster for the initialization. In this way, we implement an “inference while train” mode to accelerate the training. Following the mode, training and inference are conducted until the last edge cluster.

Besides, considering the limited computing power of edge clusters, we perform pruning operations before the parameter transfer, *i.e.*, cutting the headcount of MHSA and STBMHSA in STM to reduce the dimensions. In particular, we perform a pruning operation after every 3 edge clusters. Each pruning operation cuts off two heads of MHSA and STBMHSA. The effectiveness of pruning is validated in the subsequent section. This can make the model lightweight and further enhance training efficiency.

Discussion: In the training and inference of a subgraph, compute-intensive tasks related to the subgraph adversely affect the latency of responding to the requests for traffic flow prediction. A proper computation offloading scheme between edge servers in an edge cluster can be used to reduce latency. Specifically, computation offloading of edge servers in an edge cluster is denoted as a Markov decision process. In addition, an edge server can select to offload any proportion of the data to other edge servers. Hence, how to acquire an optimal computation offloading scheme to minimize overall latency per perception and decision-making capabilities, deep reinforcement learning (*e.g.*, Deep Deterministic Policy Gradient) is a promising solution for such a problem [33].

VI. EXPERIMENTS

A. Experimental Settings

Datasets: STGLLM-E is evaluated on two real-world large-scale datasets LondonHW and ManchesterHW, released by [18]. These two datasets originate from the sensors deployed on highways. In LondonHW, 1000 sensors are deployed to collect traffic flow data centered on London, covering from January 1st, 2014, to December 31st, 2014. In ManchesterHW, 1000 sensors are employed to monitor traffic flow centered on Manchester, covering the period from January 1st, 2014, to December 31st, 2014. The distribution of monitored sensors in the two large-scale road networks is shown in Fig. 7. The data collection frequencies for LondonHW and ManchesterHW are 15 minutes. Besides, the LondonHW and ManchesterHW datasets have the following

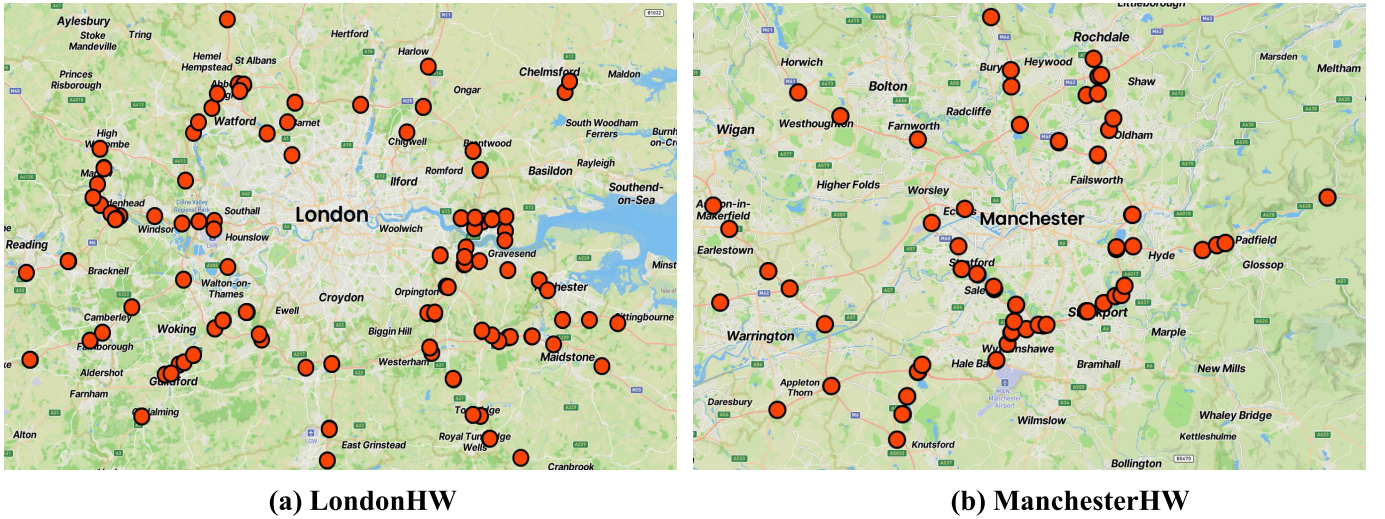


Fig. 7. Figures (a) and (b) illustrate the distribution of monitored sensors in two large-scale road networks.

TABLE II

DETAILED STATISTICS OF LONDONHW AND MANCHESTERHW. STD IS THE STANDARD DEVIATION

Dataset	Nodes	Timesteps	Mean	Std
LondonHW	1000	35040	319.52	254.81
ManchesterHW	1000	35040	270.26	228.60

differences. 1) As shown in Fig. 7, the sensors in the LondonHW dataset are distributed sparsely, while the sensors in the ManchesterHW dataset are relatively concentrated. 2) As shown in TABLE II, the traffic flow in the LondonHW is generally higher and more fluctuating compared to the ManchesterHW. We apply Z-Score normalization to normalize the data. We chronologically put the two datasets into training, validation, and test sets, with the ratio of 7:1:2.

Baselines: STGLLM-E is compared with the following advanced baselines:

- ARIMA [5]: It is a classical statistical method that integrates autoregression and moving average to model past time series data.
- LSTM [15]: It is an evolved version of Recurrent Neural Networks that adopts LSTM units to extract the temporal dependencies and predict future traffic flow at multiple timesteps.
- DCRNN [7]: It employs a bi-directional random walk on a road network with GRU in an Encoder-Decoder fashion to extract the spatio-temporal correlations.
- GWN [8]: In the baseline, GCN with the self-adaptive adjacency matrices is used to extract the spatial dependencies. Gated Temporal Convolution is adopted to uncover the temporal dependencies.
- GMAN [9]: The baseline is an Encoder-Decoder architecture. Both the Encoder and Decoder incorporate a spatial attention, a temporal attention, and a gating mechanism. Additionally, GMAN designs a transform attention to avoid the dynamic decoding between the Encoder and Decoder.

- FPT [11]: It is an unified framework, which deploys pre-trained GPT-2 with frozen multi-head self attention layers and feedforward layers for the various time series analysis tasks like imputation, classification, and long- and short-term prediction.
- LLM4TS [12]: It is a time series prediction method that leverages the pre-trained GPT-2 as the backbone. LLM4TS is trained through two-stage fine-tuning.

Metrics: The prediction accuracy and training efficiency of STGLLM-E and baselines are evaluated by five metrics. They are organized into the following two divisions: a) Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and Symmetric Mean Absolute Percentage Error (SMAPE), which are used to assess the prediction accuracy, and b) training time and GPU memory, which serve as references for evaluating the training efficiency. The five metrics are described in detail below.

1) MAE:

$$\text{MAE} = \frac{1}{Q \times n_j} \sum_{m=1}^Q \sum_{v_i=1}^{n_j} |\hat{x}_{v_i}^m - y_{v_i}^m|. \quad (22)$$

2) RMSE:

$$\text{RMSE} = \sqrt{\frac{1}{Q \times n_j} \sum_{m=1}^Q \sum_{v_i=1}^{n_j} (\hat{x}_{v_i}^m - y_{v_i}^m)^2}. \quad (23)$$

3) SMAPE:

$$\text{SMAPE} = \frac{100\%}{Q \times n_j} \sum_{m=1}^Q \sum_{v_i=1}^{n_j} \frac{|\hat{x}_{v_i}^m - y_{v_i}^m|}{(|\hat{x}_{v_i}^m| + |y_{v_i}^m|)/2}. \quad (24)$$

4) Training time: The training time contains the Total Time (TT) and the Average Time (AT). The former reflects the time overhead of the entire training process until a model attains satisfactory precision, while the latter shows the average training time of subgraphs. We can measure the training efficiency of the model through the training time. For instance, if the

accuracy of the two models is comparable, shorter training time implies more efficient training.

5) GPU Memory: The metric illustrates the GPU memory usage during the training phase. GPU Memory can evaluate the training efficiency of a model from the perspective of space overhead. For example, lower GPU Memory means fewer model parameters that need to be trained. This shows that the model does not consume excessive GPU resources during training.

Parameter Settings: In the experiment, we use the same group of parameter settings whether LondonHW or ManchesterHW. The statsmodels package in Python are employed to accomplish ARIMA. The remaining models are implemented by the PyTorch library. For STGLLM-E, several parameters are set as follows.

In graph segmentation, the number of subgraphs is set to 10 for the LondonHW and ManchesterHW datasets after repeated attempts. Accordingly, 60 Nvidia 1080 Ti GPU cards are used as 60 edge servers. We allocate certain edge servers for each subgraph based on Equation (20).

In each STGLLM, the predicted timestep Q is set to 12, *i.e.*, predicting traffic flow for the next 12 horizons with 15-minute time slots between horizons, which is the standard benchmark-setting in the domain of traffic flow prediction [7], [8], [9]. The referenced historical timesteps P are 108, in which we set $P_h = 12$ (past 3 hours), $P_d = 72$ (past 6 days at the same time period as the prediction window), and $P_w = 24$ (past 2 weeks at the same time interval as the prediction window), respectively. According to $P = 108$, we set the number of patches $P_a = 9$ and the length of each patch $L_p = 12$, respectively. For training, we choose Adam as the optimizer to minimize the MSE loss. The learning rate and batchsize are set to 0.0002 and 8, respectively. In addition, several hyperparameters contain the initial number of heads in MHSA and STBMHSA N_h , the dimensions of each head d , the number of ST blocks L , the MHSA layers in each ST block L_{MHSA} , the STBMHSA layers in each ST block $L_{STBMHSA}$, and the learning rate Lr . We fine-tune these hyperparameters on the validation set. After the repeated experiments, on LondonHW, the best performing on the setting is $N_h = 16$, $d = 4$, $L = 3$, $L_{MHSA} = 1$, $L_{STBMHSA} = 1$, $Lr = 0.0002$, and $d_{model} = N_h \times d$. On ManchesterHW, the best performing on the hyperparameter setting is consistent with LondonHW, except for the number of ST blocks $L = 2$.

In pruning, we set the initial number of heads in MHSA and STBMHSA N_h to 16. After every 4 subgraphs, we perform a pruning operation, *i.e.*, reducing the number of heads in MHSA and STBMHSA.

B. Experimental Results

The Convergence of STGLLM-E: To verify the stability of STGLLM-E, Fig. 8 shows the average convergence curve of the MSE function for all subgraphs over 500 epochs. Due to space limitations, we cannot plot the convergence curve of the MSE function on each subgraph. Hence, as shown in Fig. 8, we average the MSE function values across all subgraphs, which reflects the overall convergence of all STGLLMs. In Fig. 8 (left), STGLLMs converge rapidly in

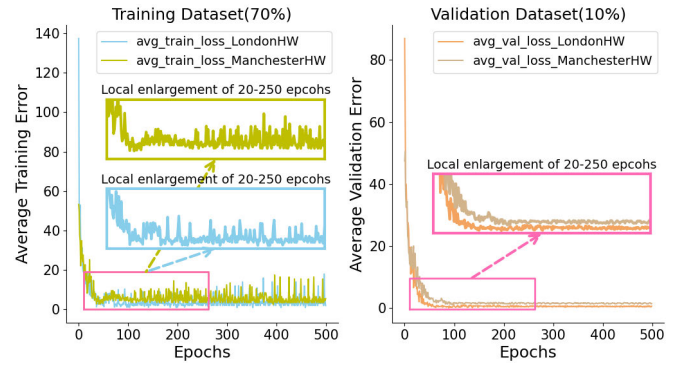


Fig. 8. Average training error (left) and average validation error (right) of all subgraphs on the LondonHW and ManchesterHW. The MSE function is treated as an error measure metric.

the training phase as a whole. In Fig. 8 (right), the average validation loss fluctuates less when the training epochs reach 100 on LondonHW and ManchesterHW. Based on these results, we draw the following conclusions. 1) STGLLMs can generally converge in less than 100 epochs. 2) STGLLM exhibits highly stable performance on each subgraph, ensuring that all STGLLMs converge simultaneously with low error rates and slightly fluctuate on the validation dataset.

Performance Comparisons: Table III compares the prediction accuracy and training efficiency of STGLLM-E and some advanced baselines for 3-horizon (3 steps), 6-horizon (6 steps), and 12-horizon (12 steps) ahead traffic flow prediction on the LondonHW and ManchesterHW datasets. In the subsequent content of this section, the 3-horizon prediction and 6-horizon prediction are characterized as the short-term prediction, while the 12-horizon prediction is described as the long-term prediction. GPUM denotes the GPU memory usage. '-' indicates that the model does not use GPU during training.

1) *Prediction Accuracy Analysis:* From the evaluation metrics MAE, RMSE, and SMAPE, we can draw the following conclusions. 1) Due to the absence of high-order manual features, the classical statistical model ARIMA is inferior to the deep learning-based time series model LSTM in prediction accuracy. 2) The results of DCRNN and GWN demonstrate that incorporating a spatial feature extraction module into the time series model accomplishes better performance. 3) A crucial finding is that spatio-temporal models perform worse than the time series model in some cases. For instance, compared with GMAN on LondonHW, the MAE, RMSE, and SMAPE values of LSTM separately decline by 28.68%, 14.37%, and 46.89% on the long-term prediction. The finding contradicts previous literatures [9]. In other words, the prediction accuracy of GMAN significantly decreases on a large-scale road network. It indicates that GMAN is inappropriate for large-scale traffic flow prediction. 4) LLM-based FPT and LLM4TS outperform spatio-temporal models like DCRNN and GWN. Specifically, in the short-term prediction, the MAE, RMSE, and SMAPE of LLM-based models are lower than spatio-temporal models. In the long-term prediction, the prediction accuracy of these models degrades compared to the short-term prediction. FPT and LLM4TS still maintain higher prediction accuracy. The commonality in the LLM-based models lies in

TABLE III

THE PREDICTION ACCURACY AND TRAINING EFFICIENCY COMPARISON OF STGLLM-E AND BASELINES ON LONDONHW AND MANCHESTERHW

Dataset	Model	3-horizon			6-horizon			12-horizon			TT	GPUM	AT
		MAE	RMSE	SMAPE	MAE	RMSE	SMAPE	MAE	RMSE	SMAPE			
LondonHW	ARIMA	286.48	429.52	42.17%	298.72	453.79	47.74%	314.72	475.74	51.47%	3.96h	-	-
	LSTM	68.31	114.83	14.28%	79.43	131.67	14.99%	95.37	165.84	16.72%	6.43h	12.49G	-
	DCRNN	55.32	91.42	12.27%	71.34	123.75	13.97%	93.72	164.72	15.69%	7.19h	5.46G	-
	GWN	43.54	77.32	12.48%	51.74	86.81	12.67%	63.26	95.86	13.75%	6.11h	4.17G	-
	GMAN	84.42	127.65	21.56%	104.58	156.94	25.83%	133.72	193.68	31.48%	5.78h	6.81G	-
	FPT	41.84	75.26	11.87%	52.37	87.11	12.36%	60.38	93.84	12.98%	2.15h	1.64G	-
	LLM4TS	39.95	74.42	12.27%	49.47	84.23	12.56%	58.46	92.42	13.32%	2.42h	1.77G	-
	STGLLM-E	38.06	72.17	10.64%	49.58	85.29	11.35%	56.23	89.63	12.16%	3.52h	2.48G	0.35h
ManchesterHW	ARIMA	245.57	380.36	46.58%	264.42	403.42	51.62%	280.15	425.45	54.84%	4.24h	-	-
	LSTM	56.46	98.78	13.95%	71.26	119.79	15.07%	83.35	141.94	16.79%	6.57h	12.93G	-
	DCRNN	38.74	74.39	13.26%	45.74	78.65	13.14%	56.42	92.35	14.03%	6.93h	6.04G	-
	GWN	40.08	75.49	12.15%	47.73	82.52	12.79%	62.44	96.75	13.85%	6.16h	4.45G	-
	GMAN	76.36	116.58	23.86%	98.45	145.35	29.64%	122.31	168.56	40.26%	6.29h	7.22G	-
	FPT	36.65	67.41	11.57%	46.64	78.25	12.38%	54.53	89.62	13.05%	2.48h	1.84G	-
	LLM4TS	34.36	67.74	12.23%	43.07	74.68	13.03%	53.35	87.36	13.92%	2.77h	2.06G	-
	STGLLM-E	34.44	68.28	11.64%	42.82	74.15	12.23%	51.76	84.48	12.75%	3.96h	2.78G	0.40h

TABLE IV

THE PERFORMANCE COMPARISON OF THREE DIFFERENT ROAD IMPORTANCE CALCULATION METHODS IN GRAPH SEGMENTATION ON LONDONHW AND MANCHESTERHW

Dataset	Model	3-horizon			6-horizon			12-horizon			TT	AT
		MAE	RMSE	SMAPE	MAE	RMSE	SMAPE	MAE	RMSE	SMAPE		
LondonHW	DC	40.25	74.03	11.57%	52.33	89.81	12.48%	59.96	93.21	13.40%	4.31h	0.43h
	PR	38.63	73.49	11.39%	50.76	88.63	12.62%	58.23	91.52	13.27%	3.67h	0.37h
	RS	38.06	72.17	10.64%	49.58	85.29	11.35%	56.23	89.63	12.16%	3.52h	0.35h
ManchesterHW	DC	35.45	71.41	13.14%	43.38	76.24	14.72%	53.95	85.62	15.47%	4.78h	0.48h
	PR	36.37	72.51	12.44%	43.56	76.13	13.98%	53.62	86.34	15.46%	4.23h	0.42h
	RS	34.44	68.28	11.64%	42.82	74.15	12.23%	51.76	84.48	12.75%	3.96h	0.40h

extensive intrinsic knowledge through pre-training, which can assist in generating more rational output sequences in the long- and short-term prediction.

STGLLM-E utilizes STM and GLLM to effectively capture the spatio-temporal correlations in the large-scale road network and generate output sequences based on extensive intrinsic knowledge, respectively. This enhances the prediction accuracy in large-scale traffic flow prediction. Hence, we observe that STGLLM-E accomplishes the best long-term and short-term prediction results on LondonHW and ManchesterHW.

2) *Training Efficiency Analysis*: From the evaluation metrics TT, AT, and GPUM on LondonHW and ManchesterHW, some following conclusions are drawn. 1) Due to the simple structure, the TT of the classical statistical model ARIMA is lower than some deep learning models (*e.g.*, LSTM, DCRNN, GWN, and GMAN). Unfortunately, the simple structure also restricts the prediction accuracy of ARIMA. 2) LLM-based FPT and LLM4TS outperform the remaining models on TT and GPUM. The primary reason is that most parameters of the former are frozen and only a few parameters need to be fine-tuned, saving training time and GPU memory usage. 3) For the metric TT, our proposed STGLLM-E is shorter than ARIMA, LSTM, DCRNN, GWN, and GMAN, and only longer than FPT and LLM4TS. The results show that STGLLM-E converges rapidly. Meanwhile, for each subgraph, the AT of STGLLM-E is only 0.35h on LondonHW and 0.40h on ManchesterHW, respectively. It indicates that our proposed model accomplishes rapid training on each subgraph. 4) Since the graph segmentation reduces the size of the large-scale road network, the GPU memory usage of STGLLM-E

significantly decreases compared with these baselines (*e.g.*, LSTM, DCRNN, GWN, and GMAN) that process the entire large-scale road network on a central server.

Ablation Study: In this section, we design several variants to evaluate the effectiveness of four parts in STGLLM-E: RoadSort in graph segmentation, STBMHSA in STGLLM, pruning in parameter transfer, and periodicity.

3) *Effect of RoadSort*: In graph segmentation, we replace our road importance calculation method RoadSort with Degree Centrality and PageRank. For a fair comparison, other parameter settings remain consistent. Table IV displays the prediction results over the next 3-horizon, 6-horizon, and 12-horizon, in which Degree Centrality, PageRank, and RoadSort are termed as DC, PR, and RS, respectively. We observe that RoadSort preserves the edges of crucial roads during graph segmentation to minimize information loss as much as possible. As a result, the model is more likely to fully extract spatio-temporal correlations and achieve more accurate long- and short-term prediction. Meanwhile, we set up an early stopping mechanism for the training on each subgraph, *i.e.*, the training is terminated early when the prediction error in the validation set reduces to the desired goal. So we find that the shorter TT and AT mean that the model converges faster.

4) *Effect of STBMHSA*: To verify the effectiveness of STBMHSA, we design the several variants for comparison. a) w/o STBMHSA. We remove the STBMHSA, *i.e.*, no spatial modeling. b) MHSA. STBMHSA is substituted by the standard MHSA. c) Local STBMHSA. STBMHSA is displaced by local STBMHSA, in which each target road focuses on its neighbors within 100km. The results for 3-horizon, 6-horizon,

TABLE V
THE PERFORMANCE COMPARISON OF SEVERAL VARIANTS IN STBMHSA ON LONDONHW AND MANCHESTERHW

Dataset	Model	3-horizon			6-horizon			12-horizon			TT	AT
		MAE	RMSE	SMAPE	MAE	RMSE	SMAPE	MAE	RMSE	SMAPE		
LondonHW	w/o STBMHSA	39.86	74.85	11.38%	51.49	86.94	12.15%	57.85	92.33	15.72%	2.48h	0.25h
	MHSA	40.63	74.08	12.36%	49.97	85.72	13.37%	57.46	90.14	14.35%	4.54h	0.45h
	Local STBMHSA	41.52	73.52	13.15%	50.04	88.43	13.56%	56.99	89.71	14.25%	3.62h	0.36h
	STBMHSA(25-50)	38.06	72.17	10.64%	49.58	85.29	11.35%	56.23	89.63	12.16%	3.52h	0.35h
	STBMHSA(25)	38.94	72.89	11.56%	51.20	87.35	12.34%	56.74	90.56	13.27%	3.25h	0.33h
	STBMHSA(25-50-100)	37.36	71.93	10.68%	49.47	85.32	11.27%	56.84	90.41	12.03%	4.11h	0.41h
ManchesterHW	w/o STBMHSA	36.36	70.77	12.19%	44.52	78.59	16.58%	54.35	88.74	20.13%	2.67h	0.27h
	MHSA	35.79	68.79	12.64%	44.27	76.74	13.84%	53.28	86.35	14.89%	5.14h	0.51h
	Local STBMHSA	34.92	69.83	13.36%	44.95	76.19	13.24%	52.73	87.26	15.02%	4.17h	0.42h
	STBMHSA(25-50)	34.44	68.28	11.64%	42.82	74.15	12.23%	51.76	84.48	12.75%	3.96h	0.40h
	STBMHSA(25)	35.13	70.14	11.69%	43.18	77.37	13.36%	53.29	85.59	14.53%	3.75h	0.38h
	STBMHSA(25-50-100)	34.06	67.99	11.84%	42.96	74.09	12.48%	51.73	84.35	13.02%	4.56h	0.46h

and 12-horizon ahead prediction are illustrated in the upper and middle portions of Table V. Some conclusions are drawn. 1) We observe that removing STBMHSA causes a degradation in long- and short-term prediction accuracy. This reveals the significance of spatial dependencies. 2) STBMHSA acquires lower long- and short-term prediction errors and shorter training time than MHSA and local STBMHSA on LondonHW and ManchesterHW. The benefit implies that STBMHSA has great strength to be a basic component for extracting the spatial dependencies within traffic flow data in real applications.

As shown in the lower portion of Table V, the different settings of the square target board are discussed. $d_1 - d_2 - d_3$ means dividing the space by three squares. The closest distance of each square from the target road is d_1 , d_2 , and d_3 km, respectively. In contrast to the 1-square dividing with $d_1 = 25$ km, the 25-50 accomplishes higher long- and short-term prediction accuracy on LondonHW and ManchesterHW due to a larger receptive field. Despite the highest long- and short-term prediction accuracy of the 25-50-100, it has a longer training time. For balance, the 25-50 is selected as the default setting.

To further discuss STBMHSA, we conduct a case study on the 25-50 square target board, centered by road LM145 on LondonHW and road AL769 on ManchesterHW. The results are shown in Fig. 9. We observe that the attention weights are dispersed when road LM145 is the target road. In contrast, the attention weights are concentrated when road AL769 is the target road. It is proven that STBMHSA is not only effective but also interpretable.

5) *Effect of Pruning*: To find the optimal pruning strategy, we design different pruning operations for selection and evaluation. a) w/o pruning. The pruning operation is removed. b) Equal Pruning-1 (EP-1). Each pruning operation cuts off one head of MHSA and STBMHSA. c) Equal Pruning-2 (EP-2). Each pruning operation cuts off two heads of MHSA and STBMHSA. d) Incremental Pruning (IP). The first pruning removes one head of MHSA and STBMHSA. The number of the removed heads in each pruning is an arithmetic sequence with one increment. Table VI illustrates the performance of comparison for the next 3-horizon, 6-horizon, and 12-horizon prediction in different pruning strategies. We find that EP-1 achieves the highest long- and short-term prediction accuracy, but its training efficiency is low. Compared with EP-1, the

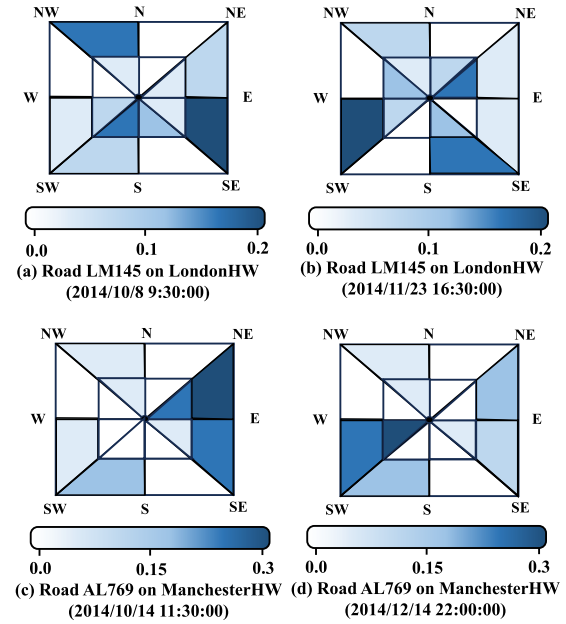


Fig. 9. Visualization of STBMHSA at the first ST block. The attention weight of the target road itself is omitted.

long- and short-term prediction accuracy of EP-2 is slightly degraded, but the training efficiency is significantly improved. Despite the optimal training efficiency of IP, its long- and short-term prediction accuracy is significantly decreased. The potential reason is that the simple structure cannot extract in-depth spatio-temporal correlations. Considering the limited computational power of the edge cluster, we choose EP-2 for pruning.

6) *Effect of Periodicity*: To assess the effect of introducing periodicity on prediction accuracy, we develop several variants for comparison. a) w/o P_d & P_w . The *daily-periodic* segment and the *weekly-periodic* segment are removed in each subgraph representations. b) w/o P_w . We exclude the *weekly-periodic* segment in each subgraph representations. c) w/o P_d . The *daily-periodic* segment is eliminated in each subgraph representations. d) We obtain the *recent* segment, the *daily-periodic* segment, and the *weekly-periodic* segment in each subgraph representations. Table VII illustrates the prediction accuracy of these variants for 3-horizon, 6-horizon, and 12-horizon ahead traffic flow prediction on the LondonHW and ManchesterHW.

TABLE VI
THE PERFORMANCE COMPARISON OF DIFFERENT PRUNING STRATEGIES ON LONDONHW AND MANCHESTERHW

Dataset	Model	3-horizon			6-horizon			12-horizon			TT	AT
		MAE	RMSE	SMAPE	MAE	RMSE	SMAPE	MAE	RMSE	SMAPE		
LondonHW	w/o pruning	43.87	78.26	12.95%	54.46	92.73	14.18%	63.24	97.29	16.52%	4.06h	0.41h
	EP-1	37.98	72.06	10.73%	49.59	84.37	11.33%	56.17	90.39	12.07%	3.75h	0.38h
	EP-2	38.06	72.17	10.64%	49.58	85.29	11.35%	56.23	89.63	12.16%	3.52h	0.35h
	IP	40.75	74.82	11.38%	52.59	89.41	12.85%	58.42	93.56	13.51%	3.23h	0.32h
ManchesterHW	w/o pruning	37.96	73.84	12.69%	47.51	79.44	14.08%	55.97	87.39	15.82%	4.66h	0.47h
	EP-1	34.31	68.19	11.52%	42.85	73.52	12.07%	51.69	84.98	13.01%	4.22h	0.42h
	EP-2	34.44	68.28	11.64%	42.82	74.15	12.23%	51.76	84.48	12.75%	3.96h	0.40h
	IP	36.17	70.38	12.26%	45.84	76.35	14.17%	52.42	86.45	16.70%	3.67h	0.37h

TABLE VII
THE PREDICTION ACCURACY COMPARISON OF SEVERAL VARIANTS FOR PERIODICITY ANALYSIS ON LONDONHW AND MANCHESTERHW

Dataset	Model	3-horizon			6-horizon			12-horizon		
		MAE	RMSE	SMAPE	MAE	RMSE	SMAPE	MAE	RMSE	SMAPE
LondonHW	w/o P_d & P_w	43.38	79.94	13.74%	56.45	91.62	14.82%	67.51	98.58	15.98%
	w/o P_w	40.69	75.86	11.71%	52.53	87.64	13.04%	59.65	93.74	13.80%
	w/o P_d	40.37	74.58	11.82%	52.67	87.24	13.49%	58.85	92.97	14.25%
	P_d & P_w	38.06	72.17	10.64%	49.58	85.29	11.35%	56.23	89.63	12.16%
ManchesterHW	w/o P_d & P_w	38.73	73.24	13.85%	46.21	78.55	14.74%	56.02	87.40	15.54%
	w/o P_w	36.58	70.41	12.49%	44.17	76.64	13.11%	53.86	85.35	13.79%
	w/o P_d	36.69	70.28	12.17%	44.02	75.86	13.26%	54.38	84.99	13.23%
	P_d & P_w	34.44	68.28	11.64%	42.82	74.15	12.23%	51.76	84.48	12.75%

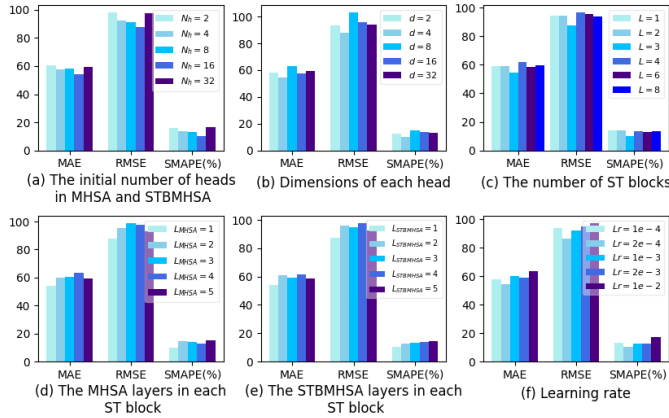


Fig. 10. Experimental results on LondonHW under different hyperparameter settings.

We find that the prediction accuracy of w/o P_w and w/o P_d outperforms w/o P_d & P_w . This reveals that learning periodicity can improve prediction accuracy. In contrast to w/o P_w and w/o P_d , P_d & P_w further enhances the prediction accuracy. The main reason is that traffic flow shows daily periodicity and weekly periodicity. For instance, due to the regularity of human daily life, traffic flow exhibits similar trends, such as morning and evening peaks. In addition, traffic flow on Tuesday exhibits similar patterns to that of previous Tuesdays, but differs from weekends. Thus, it is beneficial to promote the prediction performance of STGLLM-E through simultaneously considering daily and weekly periodicity.

Hyperparameter Analysis: The MAE, RMSE, and SMAPE values of STGLLM-E under different hyperparameter settings over the next 12-horizon prediction on LondonHW and ManchesterHW are shown in Figs. 10 and 11. When adjusting one hyperparameter, the remaining hyperparameters are set to default optimal values. We apply the Bayesian optimization

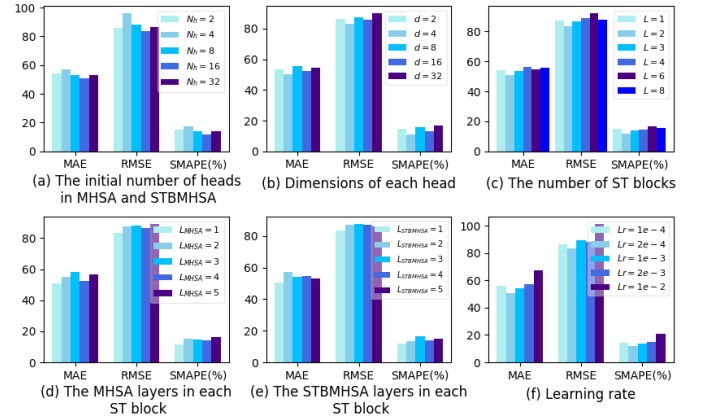


Fig. 11. Experimental results on ManchesterHW under different hyperparameter settings.

method [34] to acquire the optimal hyperparameters. In detail, by referring to literatures [35], [36], [37], we start to specify a range of possible values for each hyperparameter. Then, a set of hyperparameters is randomly selected to evaluate the prediction accuracy. Based on the evaluation results, we dynamically adjust the search space to find the optimal hyperparameter. We can get some common conclusions according to the prediction results on LondonHW and ManchesterHW. As depicted in Figs. 10 (a-c) and 11 (a-c), we observe that the smaller models are more prone to overfitting, while the larger models are more prone to underfitting. Conversely, Figs. 10 (d-e) and 11 (d-e) illustrate that the fewer layers accomplish the higher prediction accuracy. The main reason is that more layers cause cumulative errors. In Figs. 10 (f) and 11 (f), we note that large learning rates are prone to missing the optimal solution, while too small learning rates are easy to get stuck in the local optimal solution. It is crucial to set an appropriate learning rate.

There are also some differences in the prediction results between LondonHW and ManchesterHW. From

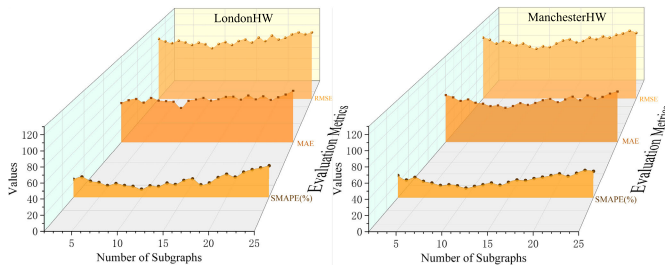


Fig. 12. The prediction accuracy comparison on LondonHW and ManchesterHW under different numbers of subgraphs.

Figs. 10 (c) and 11 (c), we find that STGLLM-E acquires the highest prediction accuracy at $L = 3$ on LondonHW, while the highest prediction accuracy occurs at $L = 2$ on ManchesterHW. The potential reason is that the optimal hyperparameter setting varies due to different characteristics in various datasets. For instance, since the characteristics of ManchesterHW are simpler, STGLLM-E with a lower complexity performs better. The characteristics of LondonHW are more sophisticated, requiring a more complex STGLLM-E to learn the features of the data.

Subgraph Segmentation Analysis: To determine the number of subgraphs in subgraph segmentation, Fig. 12 shows the MAE, RMSE, and SMAPE values of STGLLM-E under different numbers of subgraphs over the next 12-horizon prediction on LondonHW and ManchesterHW. Graph segmentation divides a large-scale road network into several subgraphs, with the number of subgraphs increasing from 2 to 25. We acquire the highest prediction accuracy until the number of subgraphs reaches 10. When the number of subgraphs exceeds 10, the prediction accuracy gradually decreases. The underlying reason is that more frequent subgraph segmentation means that more edges connecting to roads are cut off, resulting in the loss of valuable information.

VII. CONCLUSION

In this paper, we have proposed a novel LLM and edge computing-based architecture named STGLLM-E for the large-scale traffic flow prediction in 6G-IATS. First, a large-scale road network has been represented as an undirected graph. To scale down the graph, we have designed a method named RoadSort to segment the graph into several subgraphs. For each subgraph, we have presented an LLM-based method named STGLLM. Expressly, STM has been proposed to capture the spatio-temporal correlations. GLLM with pre-trained GPT-2 as a backbone has been incorporated to predict the future traffic flow. Second, to improve the training efficiency of STGLLM-E, we have developed an edge training strategy leveraging edge computing. STGLLM-E has indicated the effectiveness by evaluating on the real-world large-scale datasets LondonHW and ManchesterHW.

Some studies have demonstrated that traffic flow is influenced by several external factors (e.g., weather and accidents). Specifically, adverse weather conditions such as heavy rain, and snowstorms, reduce road capacity and increase travel time, significantly affecting traffic flow. Hence, future work will further improve the performance of traffic flow prediction by considering these external factors.

REFERENCES

- [1] Y. Liu, L. Huo, J. Wu, and A. K. Bashir, "Swarm learning-based dynamic optimal management for traffic congestion in 6G-driven intelligent transportation system," *IEEE Trans. Intell. Transp. Syst.*, vol. 1, no. 1, pp. 1–16, May 2023.
- [2] H. Sedjelmaci, N. Kaaniche, A. Boudguiga, and N. Ansari, "Secure attack detection framework for hierarchical 6G-enabled Internet of Vehicles," *IEEE Trans. Veh. Technol.*, vol. 73, no. 2, pp. 2633–2642, Feb. 2024.
- [3] G. Zheng, W. K. Chai, and V. Katos, "A dynamic spatial-temporal deep learning framework for traffic speed prediction on large-scale road networks," *Expert Syst. Appl.*, vol. 195, Jun. 2022, Art. no. 116585.
- [4] J. Tang, F. Liu, Y. Zou, W. Zhang, and Y. Wang, "An improved fuzzy neural network for traffic speed prediction considering periodic characteristic," *IEEE Trans. Intell. Transp. Syst.*, vol. 18, no. 9, pp. 2340–2350, Sep. 2017.
- [5] M.-C. Tan, S. C. Wong, J.-M. Xu, Z.-R. Guan, and P. Zhang, "An aggregation approach to short-term traffic flow prediction," *IEEE Trans. Intell. Transp. Syst.*, vol. 10, no. 1, pp. 60–69, Mar. 2009.
- [6] L. Vanajakshi and L. R. Rilett, "A comparison of the performance of artificial neural networks and support vector machines for the prediction of traffic speed," in *Proc. IEEE Intell. Vehicles Symp.*, May 2004, pp. 194–199.
- [7] Y. Li, R. Yu, C. Shahabi, and Y. Liu, "Diffusion convolutional recurrent neural network: Data-driven traffic forecasting," 2017, *arXiv:1707.01926*.
- [8] Z. Wu, S. Pan, G. Long, J. Jiang, and C. Zhang, "Graph WaveNet for deep spatial-temporal graph modeling," 2019, *arXiv:1906.00121*.
- [9] C. Zheng, X. Fan, C. Wang, and J. Qi, "GMAN: A graph multi-attention network for traffic prediction," in *Proc. AAAI Conf. Artif. Intell.*, Apr. 2020, vol. 34, no. 1, pp. 1234–1241.
- [10] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language models are unsupervised multitask learners," *OpenAI Blog*, vol. 1, no. 8, p. 9, 2019.
- [11] T. Zhou, P. Niu, L. Sun, and R. Jin, "One fits all: Power general time series analysis by pretrained LM," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 36, 2024, pp. 1–24.
- [12] C. Chang, W.-Y. Wang, W.-C. Peng, and T.-F. Chen, "LLM4TS: Aligning pre-trained LLMs as data-efficient time-series forecasters," 2023, *arXiv:2308.08469*.
- [13] U. Brandes, "A faster algorithm for betweenness centrality," *J. Math. Sociol.*, vol. 25, no. 2, pp. 163–177, Dec. 2001.
- [14] S. Brin, "The pagerank citation ranking: Bringing order to the web," in *Proc. ASIS*, vol. 98, 1998, pp. 161–172.
- [15] R. Fu, Z. Zhang, and L. Li, "Using LSTM and GRU neural network methods for traffic flow prediction," in *Proc. 31st Youth Acad. Annu. Conf. Chin. Assoc. Autom. (YAC)*, Nov. 2016, pp. 324–328.
- [16] J. Atwood and D. Towsley, "Diffusion-convolutional neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 29, 2016, pp. 1–28.
- [17] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 1–22.
- [18] C. Wang et al., "PFNet: Large-scale traffic forecasting with progressive spatio-temporal fusion," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 12, pp. 14580–14597, Dec. 2023.
- [19] Y. Chen, X. Wang, and G. Xu, "GATGPT: A pre-trained large language model with graph attention network for spatiotemporal imputation," 2023, *arXiv:2311.14332*.
- [20] S. Zhang, S. Bao, K. Chi, K. Yu, and S. Mumtaz, "DRL-based computation rate maximization for wireless powered multi-AP edge computing," *IEEE Trans. Commun.*, vol. 1, no. 1, pp. 1–17, Apr. 2023.
- [21] K. Toczé and S. Nadjm-Tehrani, "A taxonomy for management and optimization of multiple resources in edge computing," *Wireless Commun. Mobile Comput.*, vol. 2018, no. 1, pp. 11, Jan. 2018.
- [22] M. Satyanarayanan, P. Bahl, R. Caceres, and N. Davies, "The case for VM-based cloudlets in mobile computing," *IEEE Pervasive Comput.*, vol. 8, no. 4, pp. 14–23, Oct. 2009.
- [23] Y. Ju et al., "NOMA-assisted secure offloading for vehicular edge computing networks with asynchronous deep reinforcement learning," *IEEE Trans. Intell. Transp. Syst.*, vol. 25, no. 3, pp. 2627–2640, Mar. 2024.
- [24] M. Kerper, C. Wewetzer, A. Sasse, and M. Mauve, "Learning traffic light phase schedules from velocity profiles in the cloud," in *Proc. 5th Int. Conf. New Technol., Mobility Secur. (NTMS)*, May 2012, pp. 1–5.
- [25] W. Long et al., "Unified spatial-temporal neighbor attention network for dynamic traffic prediction," *IEEE Trans. Veh. Technol.*, vol. 72, no. 2, pp. 1515–1529, Feb. 2023.

- [26] J. Zhang, Y. Zheng, J. Sun, and D. Qi, "Flow prediction in spatio-temporal networks based on multitask deep learning," *IEEE Trans. Knowl. Data Eng.*, vol. 32, no. 3, pp. 468–478, Jan. 2019.
- [27] Q. Lai, J. Tian, W. Wang, and X. Hu, "Spatial-temporal attention graph convolution network on edge cloud for traffic flow prediction," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 4, pp. 4565–4576, Apr. 2023.
- [28] R. Liu, L. Gao, and J. Wu, "Key nodes mining in transport networks based in PageRank algorithm," in *Proc. Chin. Control Decis. Conf.*, Jun. 2009, pp. 4413–4416.
- [29] Z. Li, C. Chen, Y. Min, J. He, and B. Yang, "Dynamic hidden Markov model for metropolitan traffic flow prediction," in *Proc. IEEE 92nd Veh. Technol. Conf.*, Nov. 2020, pp. 1–5.
- [30] Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam, "A time series is worth 64 words: Long-term forecasting with transformers," 2022, *arXiv:2211.14730*.
- [31] K. Lu, A. Grover, P. Abbeel, and I. Mordatch, "Frozen pretrained transformers as universal computation engines," in *Proc. AAAI Conf. Artif. Intell.*, 2022, vol. 36, no. 7, pp. 7628–7636.
- [32] X. Xu, H. Zheng, X. Feng, and Y. Chen, "Traffic flow forecasting with spatial-temporal graph convolutional networks in edge-computing systems," in *Proc. Int. Conf. Wireless Commun. Signal Process. (WCSP)*, May 2020, vol. 69, no. 5, pp. 251–256.
- [33] X. Xu, C. Yang, M. Bilal, W. Li, and H. Wang, "Computation offloading for energy and delay trade-offs with traffic flow prediction in edge computing-enabled IoT," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 12, pp. 1–11, Jun. 2022.
- [34] C. E. Rasmussen, "Gaussian processes in machine learning," in *Summer School on Machine Learning*. Cham, Switzerland: Springer, 2003, pp. 63–71.
- [35] Y. Fang, F. Zhao, Y. Qin, H. Luo, and C. Wang, "Learning all dynamics: Traffic forecasting via locality-aware spatio-temporal joint transformer," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 12, pp. 23433–23446, Dec. 2022.
- [36] S. Zhang, Y. Guo, P. Zhao, C. Zheng, and X. Chen, "A graph-based temporal attention framework for multi-sensor traffic flow forecasting," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 7, pp. 7743–7758, Jul. 2022.
- [37] Y. Liang, "AirFormer: Predicting nationwide air quality in China with transformers," in *Proc. 37th AAAI Conf. Artif. Intell.*, 2023, vol. 37, no. 12, pp. 14329–14337.



Yi Rong received the B.Sc. and M.Sc. degrees from Shanghai Normal University, Shanghai, China, in 2020 and 2023, respectively. He is currently pursuing the Ph.D. degree in software engineering with the College of Computer Science and Software Engineering, Hohai University, Nanjing, China. His research interests include spatio-temporal prediction, edge computing, intelligent transportation systems, and the Internet of Vehicles.



Yingchi Mao received the B.E. and M.S. degrees in computer application technology from Hohai University, Nanjing, China, in 1999 and 2003, respectively, and the Ph.D. degree in computer software and theory from Nanjing University, Nanjing, in 2007. She is currently a Professor with the College of Computer Science and Software Engineering, Hohai University. Her research interests include edge intelligent computing, the Internet of Things data analysis, and mobile sensing systems. She is a Senior Member of CCF.



Huajun Cui received the Ph.D. degree from the School of Cyber Security, University of Chinese Academy of Sciences, Beijing, China. He is currently an Engineer with the Digital Intelligence Research Institute, PowerChina Beijing Engineering Corporation Ltd., Beijing. His current research interests include mobile edge computing, mobile application security, and 5G/6G security.



Xiaoming He (Member, IEEE) received the Ph.D. degree in computer science and software engineering from Hohai University, Nanjing, China, in 2023. He is currently a Lecturer with the College of Internet of Things, Nanjing University of Posts and Telecommunications (NJUPT), Nanjing. Before joining NJUPT, he was a Visiting Research Fellow with Singapore University of Technology and Design. His current research interests include edge intelligence and FPGA-based AI accelerators.



Mingkai Chen (Member, IEEE) received the Ph.D. degree in information and communication engineering from Nanjing University of Posts and Telecommunications, China, in 2019. He is currently an Associate Professor with Nanjing University of Posts and Telecommunications. His research interests include multimedia communications and computing, resource allocation, and signal processing in wireless networks.